

Contents

The C Piscine Final Exam Guide	2
The promise and the rule	2
Three ways through this guide.....	2
Legend.....	4
Part I — Rules of the game.....	5
What the final exam is	5
The examshell environment	6
How the Moulinette grades	8
Reading a trace, line by line	9
Part II — The offline toolkit	11
Man pages and system headers	11
Compile exactly like the grader.....	11
Debuggers: gdb and lldb as peers.....	12
Valgrind	17
Testing yourself like the Moulinette does	17
Vim survival minimum	20
Part III — The concepts, level by level	22
How these chapters work.....	22
Level 0 — Exact output.....	22
Level 1 — Strings, ASCII, pointers	26
Level 2 — Reimplementing libc, atoi, bits	32
Level 3 — Number theory, recursion, bases, malloc'd arrays	37
Level 4 — Heap arrays, sorting, lists, 2-D thinking.....	45
Level 5 — Parsers, stack machines, interpreters	52
The far side — levels 6 to 10	59
Part IV — Strategy and preparation.....	61
Exam-day playbook.....	61
Preparation plans	62
Practicing under real conditions	63
What the rushes were really teaching.....	63
Part V — Appendices	65
Appendix A — Pre-submit checklist.....	65
Appendix B — Universal edge-case matrix.....	65
Appendix C — GDB and LLDB card.....	66
Appendix D — Trace-reading card	67
Appendix E — Valgrind decoder	68
Appendix F — Master pool table	68
Appendix G — Glossary	71

42 • C PISCINE

The C Piscine Final Exam Guide

For: 42 Barcelona pisciners of every level — whether you are ten days in or chasing the top score. It covers the final exam: the solo, machine-graded test near the end of the Piscine. Preparing a weekly exam instead? Stay — the final draws on the same level-based exercise pool, just deeper.

The promise and the rule

This guide contains zero solutions to pool or module exercises. That is not caution — it is the design. Every complete program in these pages solves an invented problem that sits next to a real one: same skills, different task.

Here is why that works. You first watch an archetype — a family of exercises sharing one skill set — being built on a neighbor task, where reading does not use up an exercise you still need to practice on your own. Then you open a blank file and earn the real exercise yourself, with no text to lean on. That act of retrieval — reconstructing the idea under your own steam — is what makes it stick.

The exam rewards exactly this kind of preparation: pools evolve, campuses differ, and familiar lists go stale, so expect new exercises built on familiar archetypes — which is precisely why this guide teaches archetypes, not answers. 🏠 If you catch yourself hunting for a finished solution to memorize, you are training the wrong muscle, and this guide will keep declining to hand you one. What it will hand you: the mechanics of the exam, the offline tools, every archetype with its classic traps, and a way to verify your own work the way the grader does.

Three ways through this guide

Three itineraries, one per situation. Stops are section titles; find them in the table of contents, and do them top to bottom — each path is ordered so that every stop uses the one before it. If you are between paths, take the slower one: the exam punishes unpracticed hands more than unread pages.

Path 1 — exam in three days. You have 72 hours and one goal: pass. Do the stops in order. Worked examples and walkthroughs are for readers with time — you want mechanics, defenses, and checklist reflexes. Day one is the rules and the map; day two is the level chapters plus the compile-and-diff drills; day three is the playbook, a timed half-day run if a simulator is available, and the night-before cards. 🏠 If a stop overruns its day, move on — on this path, coverage beats depth — but a self-check you cannot answer sends you back to that chapter's Concepts section for the ten minutes it takes. Sleep the night before. Preparation plans expands this path day by day.

Stop	What to do there	Day
Rules of the game (all of Part I)	Read fully — you cannot afford a mechanics zero	1

Stop	What to do there	Day
Pre-submit checklist and Master pool table (appendices)	Skim: know the map, not every street	1
Level chapters, Level 0 through Level 3	Objectives, pitfall matrix, and self-checks only	2
Compile exactly like the grader, then Testing yourself like the Moulinette does	Terminal open; set up the compile alias and the diff habit (new to vim? read Vim survival minimum first — it is ten minutes)	2
Exam-day playbook	Read twice; internalize the submission protocol	3
Timed half-day run under real conditions (see Practicing under real conditions)	Offline, timed, one grading check per exercise	3
Night before	Re-read the Pre-submit checklist, the Universal edge-case matrix, and the Trace-reading card	3

Path 2 — a steady two weeks. The guide was built for this pace: everything, in order. The one non-negotiable is the retrieval step — after each level chapter you write that level's pool exercises yourself, cold. Reading two chapters back to back without writing any code is the classic failure mode. Days are approximate; the order is the point.

Days	Stops	What to do
1-2	Rules of the game; The offline toolkit	Terminal open; run every recipe as you read
3-10	One level chapter per day and a half, Level 0 through The far side	After each chapter, do that level's pool exercises cold — the retrieval is the actual training
11-12	Strategy and preparation; one full simulator session (see Practicing under real conditions — tool availability drifts 🏠)	Full-dress rehearsal: offline, timed, vim only, submit-once discipline
13-14	Your weakest pitfall matrices; all appendices	Re-drill and memorize the tear-out cards

Path 3 — chasing the top score. This path assumes you already clear levels 0-3 cold on first attempts. Your risks are different: hours sunk into unreachable levels, and unpracticed debugging under pressure at level 5. Spend your reading time where your points are.

Stop	What to do there	Rough time
Level 4, Level 5, and The far side	Full treatment: worked examples, walkthroughs, matrices	2-3 days
Debuggers: gdb and lldb as peers; the valgrind section	Deep fluency — the upper levels are debugging contests	1 day
Preparation plans (top-score branch)	Use the benchmarks as gates — a working L5-class parser in under 45 minutes	ongoing
Exam-day playbook	Pacing matters more when you plan to stay the full stretch	45 min

One honest note before you commit to Path 3: the marginal points live at levels 5-7, and the single account we have of the levels beyond suggests they are effectively unreachable — not worth preparation time for most. ⚠️

Whichever path you take, the two sections everyone should read twice are How the Moulinette grades and the Exam-day playbook.

Legend

Exam claims carry markers by how well they are sourced:

-  — corroborated by two or more independent accounts. Unmarked prose is corroborated too.
-  — single-source: one first-hand account reports it.
-  — campus- or era-dependent: verify on your campus.

No official exam documentation exists — every claim traces to community sources; the method and the ranked source list live in Appendix H.

Part I — Rules of the game

What the final exam is

The final exam is a solo, offline, machine-based test: you alone at a campus computer near the end of the Piscine, typing into `examshell`, the program that assigns exercises and takes your submissions. It sits after the three weekly exams (Exam00–02) and draws on the same level-based exercise pool they do — it just runs deeper. The concept sets are cumulative: Exam00 covers roughly your first days of C, Exam01 and Exam02 add `argv` handling and `malloc`, and the final adds function pointers, linked lists, and the hard parser exercises. It lasts 8 hours, twice a weekly exam — though some accounts describe a 7-hour final. 🏠

Get one thing straight before the numbers: campus variance is the norm. Scoring ladders, the maximum level, whether you receive grader reports, which editors are installed, and the exact exercise pool all differ across campuses and years. 🏠 As an illustration of how far this goes, one 2021 account from 42 Abu Dhabi describes a final scored as eight exercises worth one point each — a different format entirely. ⚠️ 🏠 And no official documentation exists to settle any of it: everything known about the exam, this guide included, comes from community accounts and can drift. Treat each specific number below as "how it usually works", then verify what you can on your own campus.

The logistics are unforgiving in one specific place. You must pre-register for the exam, and there is a short login window at the start — accounts describe about ten minutes; miss it and you fail by default. Once you are in, the world closes: no internet, no peers, no notes, no phones or notebooks. The exam contents are under NDA — a non-disclosure agreement, which is also the reason those official docs do not exist.

The mechanics, then. Everyone starts at level 0 and receives one randomly assigned exercise at their current level. Pass it and the next level unlocks; there is no skipping, so if you cannot solve any exercise at your level, you are stuck there. Fail it and you get a different exercise at the same level, worth fewer points. The points, in one table:

Scoring mechanic	Value
First attempt at a level	9 points ✓
Second attempt — a different exercise, same level	4 points ✓
Fail twice	the level is worth 0 — the third exercise only moves you forward ✓
Validating the exam	at least 25 of 100 points ✓
Three levels cleared on first attempts	roughly 27 points — already over the line ✓
Weekly-exam first attempts	reported as 16 or 20 points; one 2019 account describes a 16→11→6→1→0 ladder ⚠️ 🏠

Take the fifth row seriously: the strategy part of this guide is largely that arithmetic taken at face value. And the last row is a warning, not trivia — weekly and final scores sit on different scales, so do not calibrate one against the other.

How deep does it go? Weekly exams cap around level 5; the final reportedly continues up to level 10, a figure that comes from a single detailed 2017 account — and how far your campus's version goes is exactly the kind of thing that varies. ⚠ Time, at least, is on your side: you may leave whenever you want, and time spent in the room is not scored — the standard advice is to stay the full stretch and keep grinding levels.

Two last things. If you took a 0 on an early weekly exam, welcome to a large club: it happens to many pisciners, usually because nobody explained the submission flow, and it is not disqualifying. This guide's job is to make sure the submission flow never costs you another point. And you may have noticed this chapter listed no exercises: the pool is catalogued exactly once, in the Master pool table appendix, and tactics live in the Exam-day playbook — this part sticks to the rules of the game.

The examshell environment

Exam day starts with a login ritual. You log into the machine as `exam` (password `exam`), authenticate as yourself with `kinit your_intra_login` — Kerberos authentication; it prompts for your intra password — and then run `examshell`. That program understands essentially three commands: `subject` (or `status`) to see your current assignment, `grademe` to be graded, and `finish` to end your session.

Your session has three places that matter. `subject/` holds the assignment text and its examples. `rendu/` is your work directory, and it is a git repository. `traces/` receives the grader's reports — traces — after each grading run. The directory contract is rigid: your work goes in `~/rendu/<exercise_name>/` — `<exercise_name>.c` plus any other file the subject names — names copied exactly from the subject; a wrong directory or file name is a self-inflicted zero.

Submitting means pushing. The loop is `git add`, `git commit`, `git push`, then `grademe` and a `y` to confirm — grading happens server-side, on what you pushed, never on what is sitting unsaved in your editor. The internal git server is called Vogsphere; you will meet the name again in your traces.

Here is a first submission, end to end. Two prompts alternate below: `examshell>` means you are typing into the exam program, and a bare `$` means your ordinary shell — you move between the two (typically in separate terminals) for the whole session. The exercise, `star_line`, is invented for this guide — its subject: argument `N` prints a line of `N` stars separated by single spaces, and `0` prints an empty line. The examshell messages are invented too — yours will read differently, the commands will not — the git lines are what git actually prints, and the bracketed numbers are the guide's callouts.

```

login: exam [1]
Password:
$ kinit jdoe [2]
Password for jdoe:
$ examshell
examshell> subject [3]
  assignment: star_line (level 0)
  files: subject/star_line/
$ cat subject/star_line/subject.en.txt [4]
  (the subject: kind, names, allowed functions, examples)
$ mkdir ~/rendu/star_line && cd ~/rendu/star_line [5]
$ vim star_line.c [6]
$ git add star_line.c [7]
$ git status --short
A star_line.c
$ git commit -m "star_line"
[master 682d2f9] star_line
 1 file changed, 14 insertions(+)
$ git push
To vogsphere:jdoe/exam-repo
  d41a2c1..682d2f9 master -> master
examshell> grademe [8]
Are you sure? (y/n) y
Grading in progress...

```

1. The machine login is `exam / exam` ; your identity comes next.
2. `kinit` authenticates you as you: intra login, intra password.
3. `subject` (or `status`) shows the current assignment; the wording here is invented, but the commands are the real interface.
4. Read the subject twice before touching code: program or function? exact names? allowed functions? what do the examples show? The next section explains why each of those matters.
5. Directory and file names are copy-pasted from the subject, never typed from memory.
6. Write the program. Then notice what is missing between this step and the push: no compile with the grader's flags, no look at the output through `cat -e` (the tool that makes line ends visible — Part II teaches it). Remember that when you reach Reading a trace, line by line.
7. `git status --short` confirms exactly one staged file and nothing stray; then commit and push, because the grader sees only what lands on Vogsphere.
8. `grademe` , confirm, and the pushed commit is graded.

A few facts about the room's software, so nothing surprises you. The shell underneath is `sh` , not `bash` or `zsh` , and it starts without your usual shell config — re-set your compile alias in every terminal you open. `vim` is the safe editor assumption; graphical editors like VS Code exist on some campuses and not on others. 🏠 The man pages work offline — `man 2 write` , `man ascii` , `man isspace` , and `man -k keyword` when you have forgotten a name — and `/usr/include` is right there to browse for prototypes. The offline toolkit part of this guide turns these into a working method.

As for the toolchain: a typical campus machine runs Ubuntu 22.04 with both compilers installed, `clang 12` and `gcc 10` — and their warning sets under `-Werror` (the flag that turns every warning into a failed build; the next section explains) differ slightly, so a file can compile clean under one and fail under the other; test with the compiler your traces show. 🏠 A typical campus machine also has `gdb` and `valgrind` — the debugger and the memory checker the offline toolkit part of this

guide teaches. 🏠 `lldb` is there too — checked on both the campus workstations and the exam machines at 42 Barcelona; as with anything machine-specific, an `lldb --version` at the start of your session costs five seconds. 🏠

How the Moulinette grades

The Moulinette — 42's automated grader — is not mysterious, just strict. This section is everything it does with your push, in order; Reading a trace, line by line shows the report it writes about it.

First gate: compilation. Your file is compiled with `-Wall -Wextra -Werror` (real traces show the exact order `-Wextra -Wall -Werror`), so any warning is a compile failure — and a compile failure is a 0 for that exercise. What is not checked matters too: `norminette` (the Piscine's style checker) is not enforced in exams and the 42 header is not required; the hard gates are clean compilation and byte-exact behavior. Grading is binary per exercise — `Grade: 1` or `Grade: 0` — and the smallest output mismatch means the 0.

Before compiling anything, the grader collects your files from Vogsphere — every file you pushed, all of which it lists in the trace. Stray files (a test `main.c`, an `a.out`, `.o` files) are a leading cause of avoidable zeros. Why can a stray file be fatal? Exercises come in two kinds, and the compile step differs. For a function exercise, the Moulinette compiles your file together with its own `main.c` — traces show lines like `gcc -Wextra -Wall -Werror sort_list.c main.c -o user_exe`
`gcc -Wextra -Wall -Werror sort_list.c main.c -o user_exe` — so a leftover test `main` of yours collides with the grader's and the build dies with a duplicate-symbol error. For a program exercise, your file is compiled alone and must provide `main` itself.

Here, stated once for the whole guide, is the answer to the question every pisciner asks: Allowed functions are listed per subject; the subject header is absolute. For a FUNCTION exercise you may test locally with your own `main` (and any convenient functions like `printf` inside it), but that test `main` must not be submitted. Whichever kind it is, the code you submit may only call what the subject allows. That covers a function exercise's file as much as a program's: if `printf` or `atoi` is not listed, reimplement what you need. If you want a sense of how seriously the school takes allowed-functions lists, every C module subject spells it out in its instructions — in C00's words: "Using a forbidden function is considered cheating. Cheaters receive a grade of -42, which is non-negotiable." That is the modules' written rule, not a documented exam rule — take it only as the reason to treat the subject's header as law.

Then your program runs. The grader renames your binary to a random string (something like `./pk3q8xv0d2m5j7wt1c9h`), so your code cannot make decisions based on `argv[0]`. Each test feeds your program a fixed invocation and compares output literally: `diff -U 3 user_output_testN testN.output | cat -e`, ending in `Diff OK :D`, or in the full diff followed by `Diff KO :(. Byte-exact means exactly that: a missing trailing newline, one stray space, an extra blank line — each is a full mismatch, worth precisely as much as printing nothing at all. On top of that, the Moulinette reportedly also checks for leaks on exercises that use malloc — write leak-free code regardless; it is cheap insurance. ⚠️`

Last: what a failed `grademe` costs you. Every failed attempt drops you down the 9→4→0 ladder, and one 2021 account reports an exponential cooldown between attempts — roughly 12 minutes, then 40 minutes, then an hour and a half. ⚠️ So never use `grademe` as your debugger: test locally, submit once. The offline toolkit part of this guide exists to make that possible, and the Pre-submit

checklist (Appendix A) is the last thing you run before every grademe — a ten-line audit that catches most of the zeros this section described.

Reading a trace, line by line

After each grading run, a report — the trace — appears in `traces/`. Pisciners who read traces properly debug in minutes; pisciners who do not resubmit blind. One caveat before the anatomy: traces are the norm, not a promise — one campus account reports getting no clues at all about what failed, and a 2024 account says availability varies by exam. 🏠

Real traces from two campuses and eras show the same eight-part anatomy, in the same order. 🏠 The specimen below is a reconstruction — the exercise, names, hashes and dates are invented, but the shape and the verdict lines match the real format. It is the report the pisciner from the previous section gets back — the one who pushed without testing. The bracketed numbers are the guide's callouts, not part of the trace.

```
[1] exam-e2r5p13
    Linux 5.15.0-52-generic x86_64
    Thu Oct  8 14:12:31 UTC 2026
    gcc (Ubuntu 10.5.0-1ubuntu1~22.04) 10.5.0
    Ubuntu clang version 12.0.1

[2] Collecting user files from Vogsphere
    git@vogsphere:jdof/exam-repo

[3] 682d2f9c7c8355eb79d3df5d9399b4a5392701f3 - jdof, Thu Oct 8 14:09:55 2026 +0000 : star_line

[4] .:
    total 4
    drwxr-xr-x 2 user user 4096 Oct  8 14:12 star_line

    ./star_line:
    total 4
    -rw-r--r-- 1 user user 312 Oct  8 14:12 star_line.c

[5] $> clang -Wextra -Wall -Werror star_line.c -o user_exe

[6] $> ./pk3q8xv0d2m5j7wt1c9h 0
[7] $> diff -U 3 user_output_test1 test1.output | cat -e
    Diff OK :D

[6] $> ./pk3q8xv0d2m5j7wt1c9h 3
[7] $> diff -U 3 user_output_test2 test2.output | cat -e
    --- user_output_test2      2026-10-08 14:12:29.113402000 +0000$
    +++ test2.output           2026-10-08 14:12:29.113402000 +0000$
    @@ -1 +1 @@$
    -* * *$
    \ No newline at end of file$
    +* * *$
    Diff K0 :(

[8] Grade: 0
    = Final grade: 0 =
```

What each part is, and what you do with it:

1. **Host info** — `hostname`, `uname -msr`, the date, and the gcc and clang versions. This tells you exactly which compilers judge you; match them locally when a warning seems to appear only on the grader's side, because the two compilers' warning sets differ. 🏠
2. **Collection** — Collecting user files from Vogsphere, plus your repository. If your fix never got pushed, it never got graded.
3. **Your git log** — the full history of your exam repo. Check that the commit you think you submitted is the last one listed.
4. **A listing of everything collected** — `ls -lAR` over your files. Spot your stray files here: a test `main.c` or an `a.out` in this listing is about to cost you.
5. **The verbatim compile line** — exactly how your file was built. Copy it into your terminal character for character; reproducing the grader's build is step one of any post-mortem.
6. **Each test's invocation** — your program run with its exact `argv`, under a randomized binary name, so behavior cannot depend on `argv[0]`. Re-run any failing invocation locally with the same arguments.
7. **The verdict per test** — `diff -U 3 user_output_testN testN.output | cat -e`, then Diff OK :D or the diff and Diff KO :(. Decoder: - lines are your output, + lines are the expected output, and \$ marks every line end; the @@ line only locates the difference — read the -/+ lines.
8. **Grades** — Grade: 0 or Grade: 1 per exercise, and = Final grade: N = at the end; binary, no partial credit.

Now read the failure in part 7 as if the grader wrote it for you — it did. The - line (yours) and the + line (expected) look identical: * * *. The tell is \ No newline at end of file under the - line: your output stopped dead after the last star, while the expected output closes the line. In a trace diff, a missing final newline shows as a \ No newline at end of file line; in your own line; in your own `./prog | cat -e` check, it shows as a missing \$. And notice that test 1 passed: the 0 case owes just an empty line, and that path printed its newline — the bug lived only in the star-printing path. A passed test proves one input worked — nothing more.

The same eight parts, compressed to a card for exam day, are Appendix D, the Trace-reading card — same numbering as here. Everything else this guide teaches about testing exists so that your first read of a trace is confirmation, not news: the offline toolkit shows how to run the same compile line, the same diffs, and the same leak checks before the Moulinette does.

Part II — The offline toolkit

Everything in this part runs on the exam machine with no network. Learn these tools before exam day; during the exam they are the difference between staring at a wrong answer and taking it apart.

Man pages and system headers

The exam room has no internet, but the machine's own documentation is complete: the `man` system works offline, and every prototype your code needs sits in `/usr/include`, readable with the same tools you use on your code.

`man` is split into numbered sections, and the number matters. Section 2 is system calls, section 3 is C library functions, section 1 is shell commands. Type `man write` with no number and you land on `WRITE(1)` — a terminal-messaging command, not the system call you meant. `man 2 write` gets you the real page, with the exact prototype and the `#include <unistd.h>` line to copy.

```
man 2 write      # the system call: prototype, headers, return value
man 3 malloc     # C library: malloc, free, calloc on one page
man ascii       # the full ASCII table in octal, decimal, and hex
man isspace     # opens the isalpha(3) family page: every is* test
man -k alpha    # forgot a name? keyword-search all pages (same as apropos)
```

`man ascii` alone is worth memorizing as a habit: it answers every "what is the code for 'a'?" question without guessing. When you page through a man page, `space` scrolls, `/word` searches, `q` quits.

The headers themselves settle arguments the pages don't. Two searches worth keeping:

```
grep -n "define INT_MAX" /usr/include/limits.h
grep -n " strcmp" /usr/include/string.h
```

The first prints the line defining `INT_MAX` as `2147483647`, prefixed with its line number. The line just above that one in the file (add `-B1` to the `grep` to print it too) defines `INT_MIN` as `(-INT_MAX - 1)`, which is the whole `INT_MIN` story in one line: the positive version of it does not fit in an `int`. The second prints the line carrying the `strcmp` prototype — `extern int strcmp (const char *__s1, const char *__s2)`, followed by some compiler attributes — useful when a subject tells you to match the behavior of a function from `libc`, the standard C library.

Compile exactly like the grader

The Moulinette (42's automated grader) compiles with `-Wall -Wextra -Werror`, so a single warning is a failed build and a zero for that exercise. Your local compile line must be at least as strict, plus `-g` so the debuggers in the next section can show you file names and line numbers:

```
cc -Wall -Wextra -Werror -g strip_letter.c -o strip_letter
```

Type that forty times and you will typo it once. Alias it instead — and remember that an alias lives only in the shell you typed it into, and the exam's `sh` starts without your usual bash or zsh config: re-set the alias in every new terminal. In plain `sh` that looks like this:

```
alias mycc='cc -g -Wall -Wextra -Werror'
mycc strip_letter.c -o strip_letter
```

A typical campus machine carries both `gcc` and `clang`, and their warning sets under `-Werror` differ slightly — a file can build clean under one and die under the other, so compile with both and trust the compiler your traces show. 🏠 Also check what `cc` points to on your machine (`cc --version`); it is one of the two.

For local testing only, add the sanitizers. At run time they catch buffer overflows and many kinds of undefined behavior — signed overflow, bad shifts, null or misaligned pointers — and name the exact line. Not every kind, though: a read of uninitialised memory passes them without a report, and that one is `valgrind`'s job. The build line:

```
cc -Wall -Wextra -Werror -g -fsanitize=address,undefined file.c -o test_san
```

Never mix a sanitizer build with `valgrind` — run them separately on separate binaries.

For a function exercise, the Moulinette compiles your file together with its own `main.c` — which is why a leftover test `main` in your submission kills the build. Reproduce the disaster locally with a two-file compile:

```
$ gcc -Wall -Wextra -Werror add_squares.c grader_main.c -o user_exe
/usr/bin/ld: in function `main':
grader_main.c:(.text+0x0): multiple definition of `main'; add_squares.c:(.text+0x20): first defined here
collect2: error: ld returned 1 exit status
```

That multiple definition of `'main'` line is exactly what a forgotten test `main` earns you on grading day. Delete yours and the same two-file compile links and runs.

You will not need Makefiles in the exam — compile directly.

Debuggers: `gdb` and `lldb` as peers

A debugger turns "it segfaulted" into "line 12, `src` is NULL" in about a minute. This guide treats `gdb` and `lldb` as first-class peers: every move below exists in both, and the card in Appendix C maps them one to one. A typical campus machine has `gdb` installed. 🏠 `lldb` is there too, on the campus workstations and the exam machines alike (checked at 42 Barcelona) — use whichever reads better to you; `gdb` stays the baseline everything here assumes. 🏠

The specimen

You learn a debugger by hunting, so here is prey. The following program is an invented task, not an exam exercise: it copies `argv[1]` into a malloc'd buffer with every occurrence of a given letter (first character of `argv[2]`) removed, so `./strip_letter banana a` should print `bnn`. Four bugs are

hiding in it. Save it as `strip_letter.c` — line numbers below refer to this listing, counting the first `#include` as line 1.

```
#include <unistd.h>
#include <stdlib.h>

char    *strip_letter(char *src, char cut)
{
    char    *out;
    int     len;
    int     i;
    int     j;

    len = 0;
    while (src[len])
        len++;
    out = malloc(len);
    if (!out)
        return (NULL);
    i = 0;
    j = 0;
    while (i < len - 1)
    {
        if (src[i] != cut)
        {
            out[j] = src[i];
            j++;
        }
        i++;
    }
    out[j] = '\0';
    return (out);
}

int main(int argc, char **argv)
{
    char    *clean;
    int     i;

    (void)argc;
    clean = strip_letter(argv[1], argv[2][0]);
    if (!clean)
        return (1);
    i = 0;
    while (clean[i])
        i++;
    write(1, clean, i);
    write(1, "\n", 1);
    return (0);
}
```

Walk it top to bottom. `strip_letter` measures `src` by scanning to the terminator (lines 11-13), allocates a buffer (line 14) and checks the allocation (lines 15-16), then copies every character that is not `cut` (lines 17-27), terminates the copy (line 28) and returns it. `main` casts `argc` to `void` (line 37 — remember that cast), calls the worker on `argv[1]` and `argv[2][0]` (line 38), measures the result the same scanning way, and prints it with `write` plus a newline. It compiles `clean` under `cc -Wall -Wextra -Werror -g` — with both `gcc` and `clang` — and it runs:

```
$ ./strip_letter banana a | cat -e
bnn$
$ ./strip_letter world o | cat -e
wrl$
```

The first answer is right. The second is not (`wrl`d was owed). And two more bugs have not even surfaced yet. `-Werror` caught none of the four — the compiler checks form, not meaning. That is what the tools below are for.

Hunt 1: the segfault and the backtrace

Run it with no arguments:

```
$ ./strip_letter
Segmentation fault (core dumped)
```

Do not reread your code hoping for inspiration — ask the debugger where it died:

```
$ gdb ./strip_letter
(gdb) run
Program received signal SIGSEGV, Segmentation fault.
0x000055e53af18198 in strip_letter (src=0x0, cut=83 'S') at strip_letter.c:12
12      while (src[len])
(gdb) bt
#0  0x000055e53af18198 in strip_letter (src=0x0, cut=83 'S') at strip_letter.c:12
#1  0x000055e53af18264 in main (argc=1, argv=0x7fff36929cc8) at strip_letter.c:38
(gdb) print src
$1 = 0x0
```

Read the crash report like a sentence: it died at line 12, inside `strip_letter`, and the arguments are printed right there — `src=0x0 . NULL`. `bt` (backtrace) shows the whole call chain: frame #1 says `main` passed that `NULL` from line 38, where `argv[1]` does not exist because `argc=1` — also printed for free. Even `cut=83 'S'` is a clue: that byte is garbage read from beyond the argument array. Half of all segfault hunts end right here, in the argument values of the first frame. The fix is one guard, replacing the cast that silenced the unused-

`argc` warning instead of using it: delete the `(void)argc;` line and open `main` with `if (argc != 3)` returning 1. What the guard prints is the subject's call, not yours: this invented subject names no output for a wrong count, so this guard prints nothing, but when a real subject names one (a lone newline, say), the guard writes exactly that before it returns.

In `lldb` the same hunt is the same three moves (comments describe what you will see):

```
$ lldb ./strip_letter
(lldb) run                # stops on the segfault at the faulting line
(lldb) bt                 # same two frames: strip_letter, then main
(lldb) frame variable src  # shows src is NULL
```

Hunt 2: the wrong answer and the breakpoint

With the guard in, `world o` still prints `wrl` instead of `wrld` — and note that `banana a` printing the right answer proved nothing; one passing test never does. No crash, so plant a breakpoint where the copy finishes (line 28) and inspect the state:

```
$ gdb ./strip_letter
(gdb) break strip_letter.c:28
(gdb) run world o
Breakpoint 1, strip_letter (src=0x7ffec556e15e "world", cut=111 'o') at strip_letter.c:28
28         out[j] = '\0';
(gdb) print i
$1 = 4
(gdb) print len
$2 = 5
(gdb) print src[i]
$3 = 100 'd'
```

The loop stopped with `i` at 4 while `len` is 5, and `src[i]` is the `'d'` that never got copied. Look back at line 19: `while (i < len - 1)` quits one character early. The fix is the loop bound: `while (i < len - 1)` becomes `while (i < len)`.

When you cannot see who is changing a variable, set a watchpoint instead of a breakpoint — the program halts on every write to it. Watching `j` in that same session shows each copy as it happens (output trimmed):

```
(gdb) break strip_letter.c:19
(gdb) run world o
(gdb) watch j
Hardware watchpoint 2: j
(gdb) continue
Hardware watchpoint 2: j
Old value = 0
New value = 1
strip_letter (src=0x7ffec556e15e "world", cut=111 'o') at strip_letter.c:26
```

In `lldb`: `b strip_letter.c:28, run world o, then p i, p len, p src[i];` the watchpoint is `watchpoint set variable j`.

Hunt 3: the invisible overflow

Now try a string that contains no `o` at all:

```
$ ./strip_letter world z | cat -e
world$
```

Looks perfect. It is not. When nothing is removed, `j` reaches `len`, and line 28 writes the terminator at `out[len]` — one byte past the `malloc(len)` block from line 14. Nothing crashes today; it corrupts quietly and crashes some other day, on the grader's machine. This is sanitizer work. Rebuild with `-fsanitize=address,undefined` and run the same input (report trimmed):

```
$ cc -Wall -Wextra -Werror -g -fsanitize=address,undefined strip_letter.c -o san
$ ./san world z
==29113==ERROR: AddressSanitizer: heap-buffer-overflow
WRITE of size 1 at 0x502000000015 thread T0
    #0 strip_letter strip_letter.c:28
    #1 main strip_letter.c:39
0x502000000015 is located 0 bytes to the right of 5-byte region
allocated by thread T0 here:
    #0 __interceptor_malloc
    #1 strip_letter strip_letter.c:14
```

Read it as three answers: what happened (a 1-byte write just past a 5-byte block on the heap — the memory pool `malloc` hands out, as opposed to your local variables), where (line 28), and where that block was born (line 14, the `malloc`). Line numbers here include the two-line guard from Hunt 1. Valgrind reports the same bug in its own dialect — next section. The fix is the classic missing `+1` for the terminator: `malloc(len)` becomes `malloc(len + 1)`.

Hunt 4: the leak

Three fixes in, the program is correct and still wrong: `clean` is never freed. No symptom, no crash — only valgrind's exit report tells you (trimmed; produced by the full invocation the next section teaches):

```
==29196== 6 bytes in 1 blocks are definitely lost in loss record 1 of 1
==29196==    by 0x1091AE: strip_letter (strip_letter.c:14)
```

The fix is one line before `main` returns: `free(clean);`.

Four bugs, four different detectors: a backtrace, a breakpoint, a sanitizer, a leak report. That mapping — symptom to tool — is the skill.

The command card

Every move above — and the ones you will want next — exists in both debuggers: launch, run with arguments, breakpoints by function or by file and line, stepping, printing variables (plain, in hex, as strings), raw memory reads, backtraces and frames, watchpoints. The full two-column card mapping each `gdb` command to its `lldb` twin is Appendix C — built to be open beside you, so it is not repeated here.

Both have a full-screen mode when you tire of `list`: `gdb -tui` (or `Ctrl-x a` inside a session) opens a source panel that follows execution, and `layout src` picks the source view; `lldb`'s is the `gui` command. Both modes share one quirk: your program's own output garbles the panel — press `Ctrl-l` to redraw.

The segfault triage drill

When a crash hits mid-exam, run this drill instead of improvising:

1. Reproduce it with the exact argument list that crashed — copy it into your shell history verbatim.
2. Rebuild with `-g` (keep `-Wall -Wextra -Werror` on).

3. `gdb ./prog`, then run with those arguments.
4. `bt`, and read the innermost frame that is in your file: note the line, then read the argument values in the frame line itself — a `0x0` there is the story half the time.
5. print the variables used on that line. If the cause is still not visible, break one line earlier, rerun, and step with `next` while printing.

Five steps, no guessing, and the crash usually names its own cause by step 4.

Valgrind

Valgrind watches every byte your program touches at run time; where the compiler checks form, valgrind checks memory conduct. One invocation is worth learning by heart, and Appendix E is its tear-out card.

```
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./prog args
```

Run against the specimen from the debugger section — before its `malloc` fix — it catches the overflow that the normal run hid (trimmed):

```
==29191== Invalid write of size 1
==29191==    at 0x10921F: strip_letter (strip_letter.c:28)
==29191== Address 0x4a8d045 is 0 bytes after a block of size 5 alloc'd
==29191==    at 0x4848899: malloc (vgpreload_memcheck)
==29191==    by 0x1091AB: strip_letter (strip_letter.c:14)
```

Same three answers as the sanitizer: what, where, and where the block was born. After the `malloc` fix but before the `free`, the same invocation reports the leak instead: 6 bytes in 1 blocks are definitely lost, again pointing at line 14. Four messages cover nearly everything you will see — Invalid write, Invalid read, definitely lost, and the uninitialised-value jump — and the decoder for all four, with what a fully clean run looks like, is Appendix E, the tear-out card. Before every submit you want valgrind's two closing lines: All heap blocks were freed and ERROR SUMMARY: 0 errors. Why insist, when the diff verdict is what scores? The Moulinette reportedly also checks for leaks on malloc-using exercises — write leak-free code regardless; it is cheap insurance. ⚠ An Invalid write today is a random crash on the grader's machine tomorrow, and that one is not "reportedly".

Testing yourself like the Moulinette does

The grader's judgment is a byte diff — nothing more forgiving. So test with byte diffs, and `grademe` becomes a formality instead of a gamble.

Start with `cat -e`, which prints `$` at every line end so invisible bytes become visible. Four outputs tell the four classic stories:

```
$ printf 'hello\n' | cat -e
hello$
$ printf 'hello' | cat -e
hello
$ printf 'hello \n' | cat -e
hello $
$ printf '\n' | cat -e
$
```

First: correct — text, then `$`. Second: missing trailing newline — no `$`, and a byte diff fails. Third: a trailing space hiding before the newline. Fourth: a lone `$`, the legitimate output of many exercises on empty input — an empty line is not the same as no output.

The grader's own verdict is produced by `diff` piped through `cat -e`, so mirror it. Write your output and the expected output to files and diff them:

```
./strip_letter "HELLO WORLD" x > got.txt
printf 'HELLO WORLD\n' > want.txt
diff -U 3 got.txt want.txt | cat -e
```

A real run of that, against a buggy build that dropped the newline (the two `---` / `+++` header lines trimmed here; your run prints them first):

```
@@ -1 +1 @@$
-HELLO WORLD$
\ No newline at end of file$
+HELLO WORLD$
```

Read it exactly as you will read a trace: `-` lines are yours, `+` lines are expected, `$` marks each line end — the full decoder is in Reading a trace, line by line. Silence from `diff` means byte-identical, which is the only passing grade. (If you type `bash` first you can shorten this to `diff <(. /mine ...) <(reference ...) | cat -e` — but process substitution is bash-only, the exam shell is `sh` where it is a syntax error, and the temp-file habit is the one to train.)

You have no reference binary in the room, but the machine ships oracles — an oracle is a trusted existing program whose output you compare your own against. `./mine` below stands for whatever you are testing:

You are writing	Oracle	The check
a letter/case cipher	<code>tr</code>	pipe the input through the equivalent <code>tr</code> sets
a string reverser	<code>rev</code>	<code>rev</code> reverses each input line
primality logic	<code>factor</code>	<code>factor 7919</code> prints <code>7919: 7919</code> — only itself: prime
arithmetic on argv	<code>expr</code> , <code>bc</code>	<code>expr 123 + 456</code> prints <code>579</code> ; <code>bc</code> handles big values
hex conversion	<code>printf</code>	<code>printf '%x\n' 255</code> prints <code>ff</code>

```

# cipher vs tr (rot13 sets shown; adapt to your subject)
./mine "Hello There" > got.txt
printf '%s\n' "Hello There" | tr 'a-zA-Z' 'n-za-mN-ZA-M' > want.txt
diff -U 3 got.txt want.txt | cat -e      # silence = byte-identical

# reverser vs rev
./mine "abcdef" > got.txt
printf '%s\n' "abcdef" | rev > want.txt
diff -U 3 got.txt want.txt | cat -e

# arithmetic vs expr and bc
expr 123 + 456                                # 579 (escape * as \*)
echo "2^31 - 1" | bc                          # 2147483647 - INT_MAX edges

# hex vs printf
printf '%x\n' 255                             # ff

```

If your reverser dropped a character, one of those diffs would show `-edcba$` against `+fedcba$` — the bug found in two seconds without reading a line of code.

For a function exercise, the oracle pattern needs a harness: your own `main.c` that feeds `argv` to your function and prints results in brackets, so empty strings and stray spaces show. This one compiles clean under both compilers with the full flag set and, run as `./try "" hello "two words"`, prints `[] -> []`, `[hello] -> [hello]`, `[two words] -> [two words]` around a stand-in that returns its input unchanged:

```

#include <stdio.h>

char    *my_function(char *s);

int main(int argc, char **argv)
{
    int i;

    i = 1;
    while (i < argc)
    {
        printf("[%s] -> [%s]\n", argv[i], my_function(argv[i]));
        i++;
    }
    return (0);
}

```

`printf` is fine here precisely because this file never ships: the test-main rules live in *How the Moulinette grades* — test with your own `main`, delete it before you push. One thing it must still avoid is handing `NULL` to `%s`, which is undefined behavior: `glibc` happens to print `(null)`, but nothing in C promises it — so if your function can return `NULL`, test for that before printing.

Then stop typing one test at a time. Feed a program the classic killers in one loop — the crashes the grader punishes most are unterminated strings, reading `argv[1]` when it does not exist, and writing past a `malloc`'d buffer. This loop (run here against the repaired specimen) puts the end of every output on screen, so bytes printed past the text show — though an unterminated string can just as well run into a zero byte and look fine, which is why the `valgrind` sweep below follows it. The second killer it never reaches: every run passes two arguments, and `""` is still an argument (`argc` counts it). Wrong counts are three runs to add by hand — `./strip_letter`

`./strip_letter`, `./strip_letter abc`, `./strip_letter abc z extra` — checking that none crashes and each does exactly what the subject says. The loop itself:

```
for a in "" " " "a" "hello world" "-2147483648" "zzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzz"; do
    printf "[%s] -> ' '$a"
    ./strip_letter "$a" z | cat -e
done

[] -> $
[ ] -> $
[a] -> a$
[hello world] -> hello world$
[-2147483648] -> -2147483648$
[zzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzz] -> $
```

Every line shows its \$, nothing crashed, the empty and all-stripped cases still print their newline. Wrap the same idea around `valgrind` for a leak sweep — this run printed `clean:` four times on the fixed build, and `PROBLEM:` four times on the leaky one:

```
for a in "" "abc" "hello world" "zzzz"; do
    if valgrind -q --leak-check=full --error-exitcode=42 ./strip_letter "$a" z >/dev/null 2>&1; then
        echo "clean: [$a]"
    else
        echo "PROBLEM: [$a]"
    fi
done
```

All of this exists to serve one law: never use `grademe` as your debugger. Every failed submission slides you down the 9→4→0 ladder, and reportedly starts a growing cooldown between attempts. ⚠️ Test locally until the diffs are silent and `valgrind` is quiet — then submit once.

Vim survival minimum

`vim` is the safe editor assumption on exam machines; graphical editors vary by campus, so being helpless in `vim` is a risk you do not need to carry. 🏠 `vim` has modes: normal mode (keys are commands), insert mode (keys are text), and command-line mode (lines starting with `:`). You press `i` to enter insert mode and `Esc` to get back to normal — when in doubt, hit `Esc`.

Twelve commands are enough to take the exam:

Keys	What it does
<code>i</code>	insert text before the cursor
<code>Esc</code>	back to normal mode
<code>:w</code>	save
<code>:q</code>	quit
<code>:wq</code>	save and quit
<code>:q!</code>	quit and discard changes
<code>dd</code>	delete (cut) the current line

Keys	What it does
yy	copy the current line
p	paste the cut/copied line below
u	undo
Ctrl-r	redo
/word	search; n jumps to the next hit

Type `:set number` first thing — the debuggers and the compiler talk to you in line numbers all day.

If vim (or your terminal) dies mid-edit, the file is not lost. A hidden swap file like `.work.c.swp` stays behind, and reopening the file shows a scary `ATTENTION/swap file found` screen. Run `vim -r work.c` to recover the unsaved state, save it with `:w`, quit, then delete the leftover with `rm .work.c.swp` so the warning stops. Try it once before exam day — kill vim mid-edit, find the swap file waiting with its owner and date, and walk the recovery — so the warning screen is a chore, not a fright, the day it matters.

Part III — The concepts, level by level

How these chapters work

Every level chapter below has the same eight parts: objectives, the concepts the level runs on, what the level actually asks, one worked example, whiteboard walkthroughs, a pitfall matrix, a self-check, and recipes to verify your own work with the tools from Part II. The order is deliberate: concepts before exercises, one full example before you write anything, verification last because it is what you will do most.

The contract is retrieval practice. Read a chapter, then close this guide and write that level's pool exercises cold — the full list per chapter is in the master pool table in Appendix F. Test each one the way the grader will (see Testing yourself like the Moulinette does), and come back to the pitfall matrix only after you have an answer to check. Reading about `ft_atoi` is not knowing `ft_atoi`; producing it from nothing, twice, is.

This guide teaches archetypes, not answers, because pools evolve, campuses differ, and new exercises appear on familiar patterns — the archetype is what transfers. 🏠 That is also why every worked example is an invented problem that is deliberately not in the pool: it exercises the same mechanics on a different task. If an exam subject looks like a worked example, do not pattern-match — re-derive; the transfer is the training.

One chapter breaks the template: the far-side chapter covers levels 6-10, which rest on a single 2017 first-hand account, so it describes difficulty classes rather than exercises. ⚠️

Level 0 — Exact output

Level 0 (L0) is the exam's front door, and nothing behind it is conceptually new — the ideas are your first days of C. The grade is decided by mechanics: exact bytes, guarded `argv`, a deliberate newline.

Objectives

- You can produce exact bytes on stdout with `write` alone, deciding every newline deliberately.
- You can explain why `'0' + 5` is a character, and print a digit without `printf`.
- You can guard `argc` before touching `argv[1]` — reflexively, every time.
- You can walk a string with a loop that stops exactly at the terminator.
- You can check your own output with `cat -e` before the Moulinette does.

Concepts

write and file descriptors. Everything L0 prints goes through `write(fd, address, count)`: file descriptor 1 is stdout, 0 is stdin, 2 is stderr. `write` wants the address of the bytes and how many to

write — never a character value where it expects an address. Which functions you may call at all is decided per subject; the rule lives in How the Moulinette grades.

Characters are small integers. 'a' is a number wearing quotes; '0' + 5 is '5' because the digit characters sit in a contiguous run, and so do 'a' - 'z' and 'A' - 'Z'. You met this on your first C days (C00), and L0 is where it pays. This demo is the whole trick:

```
#include <unistd.h>

int main(void)
{
    char    c;

    c = '0' + 5;
    write(1, &c, 1);
    write(1, "\n", 1);
    return (0);
}
```

Compiled with `cc -Wall -Wextra -Werror` and run, it prints 5 followed by a newline. Note `&c`: the address of the byte, not the byte. Printing a letter is the same two lines — assign 'a' to `c` instead and the `write` call does not change.

Loops and bounds. An argument is a C string: bytes ending in the `'\0'` terminator. The safe walk advances one byte at a time while the current byte is not the terminator — the condition is the bound, so the loop stops exactly at the end, and on an empty string it does zero passes and falls through.

argv and argc. `argv[0]` is the program's own name, real arguments start at `argv[1]`, and `argv[argc]` is NULL — which is exactly why touching `argv[1]` unguarded segfaults. You met this on the argv day (C06).

Byte-exactness. The Moulinette diffs your output against the expected bytes, and grading is binary — the smallest mismatch is a 0 for the exercise. Piping through `cat -e` makes the invisible visible: `$` marks every line end. The habit belongs in your hands before exam day; the trace-side version of the same decoder is in Reading a trace, line by line.

What the level asks

L0 is the exact-output gate: tiny programs whose whole difficulty is doing precisely what the subject shows, byte for byte. The single-character printers — `aff_a`, `aff_z`, `only_z` — emit one exact character plus, usually, a newline, some of them hunting for their letter in the input first; the attested addition `only_a` is a classic-era peer of `only_z`, seen outside the baseline enumeration. 🏠 What each prints when the input is missing or the letter absent is where the points are actually lost: the subject states the fallback, and no two subjects owe you the same one.

The whole-argument printers `aff_first_param` and `aff_last_param` print one of the program's arguments; the entire skill is guarding `argc` and picking the right index without an off-by-one. The alphabet marchers `maff_alpha` and `maff_revalpha` walk the alphabet in some direction with a case pattern that depends on position — a loop, character arithmetic, and parity.

Three more exercises belong here pedagogically even though the classic enumeration lists them at level 1, while most other sources place them at L0. 🏠 `hello` is pure exact output; `ft_countdown`

and `ft_print_numbers` are digit runs from character arithmetic and a loop whose bounds you must defend. Finally, `strlen_sh` appears in the classic list as a shell-era historical leftover, unlikely in a modern pool. 🏠

Worked example: initials

An invented program, not in the pool. Spec: print the first character of each argument, in order, then one newline; an empty argument contributes nothing; with no arguments, print just the newline.

```
#include <unistd.h>

int main(int argc, char **argv)
{
    int i;

    i = 1;
    while (i < argc)
    {
        if (argv[i][0] != '\0')
            write(1, &argv[i][0], 1);
        i++;
    }
    write(1, "\n", 1);
    return (0);
}
```

Compiled with `cc -Wall -Wextra -Werror` and run: `./initials hello wide world` prints `hww` and a newline (`hww$` under `cat -e`); `./initials` alone prints just the newline (a bare `$`); `./initials ""` zebra prints `z$` — the empty argument was skipped, not crashed on.

Line by line. `unistd.h` declares `write`. The loop starts at 1 because `argv[0]` is the program's name, and runs while `i < argc`, so with no arguments it runs zero times — the `argc` guard is built into the bound rather than bolted on. `argv[i][0]` is the argument's first byte; testing it against `'\0'` catches empty arguments, whose first byte is already the terminator. `&argv[i][0]` is that byte's address — `argv[i]` alone means the same thing, but the `&...[0]` spelling generalizes to any index. The final `write` sits outside the loop on the single exit path, so every case ends with exactly one newline.

Now go do the real ones: this is the same `argv` discipline `aff_first_param` and `aff_last_param` grade, and the same guarded-index habit `paramsum` -class exercises assume later.

Walkthroughs

The L0 skeleton — what every one of these programs decides before touching `argv[1]`:

1. Read the subject: how many arguments does this program expect?
2. If `argc` is anything else, print exactly the fallback the subject names (often just a newline — but read yours), then stop.
3. Only now touch `argv[1]`. Walk it byte by byte, stopping at the terminator.
4. Emit through `write`, one byte at a time, from the byte's address.
5. Match the subject's line-end policy exactly: does the example end with a newline or not?

The alternating-case march (maff_alpha -class reasoning):

1. Track two things: which letter you are on, and its position counted from the origin the subject uses.
2. Position parity picks the case band. The subject's example tells you which parity is uppercase – do not guess it.
3. Case is arithmetic, not lookup: the two bands are a fixed distance apart in the character table.
4. Pitfall: decide whether position means "index in the output" or "letter of the alphabet" – the subject's example disambiguates.
5. End-of-output newline policy, as always, from the example.

Pitfall and edge-case matrix

The level-specific traps, in the form the grader will serve them.

Input or situation	Classic bug	Defense
<code>./prog</code> with no arguments	Reading <code>argv[1]</code> — but <code>argv[argc]</code> is <code>NULL</code> — segfault	Guard with <code>argc</code> first (or bake the guard into the loop bound); print the subject's fallback
<code>./prog ""</code>	Assuming at least one byte exists; printing the terminator	A <code>while (s[i])</code> traversal does zero passes on <code>""</code> ; check what the subject still owes (usually the newline)
Output looks right, grade is 0	Missing trailing newline — invisible in a terminal	Pipe through <code>cat -e</code> on every run; a missing <code>\$</code> at the end is the tell
Passing a character value to <code>write</code>	<code>write(1, argv[1][i], 1)</code> treats byte 104 as an address	Pass the address, <code>&argv[1][i]</code> ; keep <code>-Werror</code> on locally so this refuses to compile
Input absent or letter not found	Hard-coding a fallback remembered from a different exercise	Each subject defines its own fallback; re-read it per exercise, print exactly that

Self-check

Q1. `cat -e` shows your output as `hww` with no `$` at the end, and the Moulinette gave a 0. What happened?

Answer

The trailing newline is missing: `$` marks a line end, and its absence means no final `\n` was written. The output text was right and the bytes were not, and grading is byte-exact — that alone is the 0.

Q2. `./prog` with no arguments segfaults. What is the first thing you check in your code?

Answer

Whether anything reads `argv[1]` before checking `argc`. With no arguments, `argv[1]` is the `NULL` that ends the `argv` array, and dereferencing it crashes. Guard first, then touch.

Q3. What is wrong with `write(1, argv[1][i], 1)`, and what is the fix?

Answer

It passes the character's value where `write` expects an address, so the byte's numeric value gets treated as a pointer. Both compilers warn about it even without flags (recent releases refuse it outright), and under `-Werror` the compile is refused. Build it anyway and it does not even crash: the kernel rejects the bogus address, `write` returns -1, and the byte is simply never printed. There is no segfault to point you at the bug, just missing output. Pass the address of the byte instead.

Q4. Without `printf`, how do you print the digit seven?

Answer

Add seven to the character zero: the digit characters are contiguous, so that arithmetic lands on the character for seven. Store it in a `char` variable and pass that variable's address to `write` with a count of one.

Q5. Two L0 subjects both involve a missing argument, and you remember one of them printing an empty line. Can you reuse that behavior in the other?

Answer

No. Fallback behavior is defined per subject and differs between exercises; assuming a universal fallback is a classic first-attempt killer. Re-read the subject you actually have and print exactly what its examples show.

Verify your work

Run every case through `cat -e`, always — normal input, no arguments, an empty-string argument:

```
./yourprog hello wide world | cat -e
./yourprog | cat -e
./yourprog "" zebra | cat -e
```

Then stop eyeballing and let `diff` judge, exactly as Testing yourself like the Moulinette does teaches. Build the expected bytes with `printf`, write both sides to files, and compare; silence means pass:

```
./yourprog hello wide world > got.txt
printf 'hww\n' > want.txt
diff -U 3 got.txt want.txt | cat -e
```

Compile with both compilers per Compile exactly like the grader before you trust a clean build. L0 exercises are also your speed drill: check yourself against the level 0–1 bar in Preparation plans.

Level 1 — Strings, ASCII, pointers

Level 1 (L1) is where strings and pointers take over: half the level reimplements small libc classics, the other half transforms `argv` text. It is also where function exercises begin, so the return value becomes part of what is graded.

Objectives

- You can reimplement a small libc function to its subject's contract — its prototype, the man page it names, and its return value included.
- You can write through a pointer parameter and explain why the caller sees the change.
- You can apply a wraparound transform inside one case band without disturbing the other.
- You can scan a string for words using a separator set, not a single character.
- You can state what your function does for the empty string — before you run it.

Concepts

Functions and their contracts. L1 is where function exercises begin: you submit a function, the Moulinette links it against its own `main` and tests it. A function's contract is its prototype and its return value — some returns carry data, some carry status, and the subject defines which, directly or through the man page it names. Copy the prototype from the subject character for character. A wrong name fails the link, but a wrong parameter or return type usually does not: C links functions by name alone, so your file can build cleanly against the grader's `main` and then return wrong values or crash. Either way the exercise is a 0, and a clean build proves nothing about the signature.

Pointers and out-parameters. C passes arguments by value: a function receives copies, and changing a copy changes nothing outside. Passing an address changes the game — the function can write through it:

```
void    store_initial(char *name, char *initial)
{
    *initial = name[0];
}
```

The caller hands over a location, and after the call that location holds the answer:

```
char    initial;

store_initial("Ada", &initial);
/* initial is now 'A' */
```

This is the mechanism the pointer days (C01) drilled: a function handed an address can change what the caller owns, and out-parameters — one address per result — are how a function returns more than one thing.

C strings. A string is a `char` array ending in the `'\0'` terminator; its length is the byte count before the terminator. Traversal is most of the level's work — advance while the current byte is not the terminator. A `strcpy`-style copy is not done until the terminator has been copied too; a bounded copy writes one only where the man page its subject names says it does — the level 2 chapter covers n-bounded semantics. The string days (C02) drilled both; the exam assumes they are reflex.

Boundaries and vacuous truth. The empty string is a real string that your loops must survive: a correct traversal does zero passes and falls through. Related and sneakier: a claim about every

character of an empty string is true — there is no character to refute it. Subjects lean on this convention; know it before it surprises you.

What the level asks

L1 splits into libc-lite reimplementations and argv transforms. On the libc side, `ft_strlen`, `ft_putstr`, `ft_strcpy`, and `ft_swap` reimplement the small classics — exact semantics from the subject, and from the man page it names when it names one, terminator handling and return values included. The prototype is always the subject's, even where libc's differs: the Piscine's `ft_strlen` returns `int`, not `size_t`. Exercises of this shape are graded as function exercises — your subject says which yours is — and your own test `main` stays home.

The transform programs take a string argument and re-emit it changed: `rot_13` shifts letters a fixed distance with wraparound, `rotone` shifts by one, `ulstr` flips each letter's case, `alpha_mirror` - style mirroring waits at the next level. `repeat_alpha` stretches letters by their alphabet position — character arithmetic again, this time as a count. `rev_print` prints its argument backwards, which is a bounds exercise in disguise: find the end, then walk down without stepping past index zero. `first_word` scans for the first word between separators; `search_and_replace` replaces every occurrence of one character with another across the string, after validating the extra arguments the subject constrains. The attested addition `fizzbuzz`, seen on a 2024 campus list, is the classic counting game: count upward, substituting words for the numbers that meet the subject's divisibility rules — loop bounds plus divisibility plus exact output. 🏠

Across all of them, two details decide grades: the newline policy when input is absent (the subject's examples show it) and treating both space and tab as separators where word exercises are concerned.

Worked example: digit halver

An invented program, not in the pool — the band-transform archetype moved off letters. Spec: given exactly one argument, print it with every decimal digit replaced by half its value, rounded down; every other byte passes through unchanged; then one newline. With any other argument count, print just the newline.

```

#include <unistd.h>

int main(int argc, char **argv)
{
    int    i;
    char   c;

    i = 0;
    if (argc == 2)
    {
        while (argv[1][i] != '\0')
        {
            c = argv[1][i];
            if (c >= '0' && c <= '9')
                c = '0' + (c - '0') / 2;
            write(1, &c, 1);
            i++;
        }
    }
    write(1, "\n", 1);
    return (0);
}

```

Compiled with `cc -Wall -Wextra -Werror` and run: `./digit_halve "a1b2"` prints `a0b1`; `./digit_halve "route 66, gate 209"` prints `route 33, gate 104`; `./digit_halve "789"` prints `344`; and `./digit_halve` with no argument prints a bare newline (`$` under `cat -e`).

Line by line. The `argc == 2` test wraps the whole transform, but the final newline sits outside it — one exit path, so the fallback case and the normal case both end in exactly one `\n` without duplicated code. Inside the loop, the byte is copied into `c`: partly so the transform reads clearly, partly because `write` needs an addressable byte to point at. The band test `c >= '0' && c <= '9'` asks "is this byte inside the digit band" — comparisons on characters are comparisons on their codes. The transform is the value pattern in one line: `c - '0'` converts a character to its value (0 through 9), `/ 2` is the arithmetic, and `'0' +` converts the value back into a character. Value out, arithmetic, value back in — never arithmetic on the raw character code: halving the code of `'7'` (55) lands on 27, a control byte, not `'3'`.

Now go do the real ones. What carries over to the pool is the pattern — value out, arithmetic, value back in — not this program: each transform subject fixes its own argument rules, the bands it touches, its mapping, and its newline policy. Working those out from the subject, band edges included, is the exercise; the walkthrough below is reasoning to check yours against, not a recipe to copy.

Walkthroughs

The wraparound cipher, in general (rot-class reasoning):

1. Classify each byte: lowercase band, uppercase band, or neither.
2. Neither: emit unchanged. Non-letters are passengers, not cargo.
3. In a band: convert to an index from the band's start, apply the shift, wrap with modulo band-size, convert back into the SAME band.
4. The two bands are two separate wraps — never one test spanning both.
5. Continue to the terminator; settle the newline from the subject.

The word scan (`first_word` -class reasoning):

1. Skip leading separators. The subject names the separator set — typically space and tab.
2. A word runs from the first non-separator until the next separator or the end of the string.
3. Emit the word's bytes as you pass them, then stop scanning.
4. Decide the newline from the subject's examples — it is usually owed even when there is no word at all.

Pitfall and edge-case matrix

Input or situation	Classic bug	Defense
Letter near the band edge (<code>z</code> , <code>y</code> under a shift)	Shifted code walks past the band's end — into punctuation for a small shift; for a larger one into the other band (<code>'z' + 13</code> is <code>'g'</code>) or out of ASCII altogether (<code>'z' + 13</code> is 135)	Work on the index-in-band, wrap with modulo, then re-enter the band
Mixed-case input	One test spanning <code>'A'</code> to <code>'z'</code> also matches the six non-letters between the bands	Two separate band tests, two separate transforms
Tab-separated words	Only space treated as a separator, so tabbed words glue together	Separator set per the subject — space and tab; test with a quoted tab
<code>search_and_replace</code> - class extra arguments	Transforming before validating the arguments' shape	Validate everything the subject constrains first; the fallback branch is part of the exercise
Copy-style function returns	Returning nothing useful, or the wrong pointer	Return exactly what the subject, or the man page it names, specifies — <code>strcpy</code> hands back the destination, <code>strncpy</code> a length; the prototype's return type is the first clue. The return value is part of the contract, and the grader's <code>main</code> is free to use it
Empty string to any of these	Special-case code for <code>""</code> that breaks something else	A correct traversal already does zero passes on <code>""</code> ; just confirm what is still owed (newline, return value)

Self-check

Q1. A shift cipher works on `"abc"` but emits punctuation for `"xyz"`. What happened?

Answer

The shift ran off the end of the letter band: adding to the raw character code walks past the last letter into the neighboring punctuation. Convert to an index within the band, add, wrap with modulo the band size, and convert back.

Q2. Why must uppercase and lowercase be handled as two separate bands?

Answer

They are two separate contiguous runs in the character table with non-letter characters between them. A single test spanning from uppercase A to lowercase z also matches those in-between bytes, and a single wrap would bleed one band into the other.

Q3. Your word scanner prints nothing at all for an input of only spaces and tabs. Is that right?

Answer

Almost certainly not quite: most word subjects still owe a newline when there is no word, so the output should usually be one empty line rather than zero bytes. The subject's examples settle it — check them, not your memory.

Q4. Your copy function copies perfectly but still fails. What contract detail do you check first?

Answer

The return value. A copy function returns exactly what its subject, or the man page the subject names, specifies — the destination pointer for `strcpy`, a length for `strncpy`; the prototype's return type is the first clue. The grader's own `main` may use whatever your function returns — a correct copy with a wrong return is a wrong function. Then check the terminator against the same contract: whether it is written at all, and where, is part of it.

Q5. What does "vacuous truth" mean for a check-every-character function handed an empty string?

Answer

A claim about every character of an empty string is true, because no character exists to violate it. So "all characters satisfy the test" conventionally answers yes for the empty string — and subjects rely on that convention.

Q6. Why can't a function swap two of the caller's variables if it receives them as plain values?

Answer

C passes arguments by value, so the function swaps its own private copies and the caller's variables never move. It needs the variables' addresses as parameters so it can write through them into the caller's memory.

Verify your work

The transform programs have ready-made oracles, exactly as laid out in *Testing yourself like the Moulinette* does. `tr` is the cipher oracle — the rot-13 mapping, a case flip, and a shift-by-one respectively:

```
printf '%s\n' "Hello" | tr 'a-zA-Z' 'n-za-mN-ZA-M'
printf '%s\n' "MiXeD case 42" | tr 'a-zA-Z' 'A-Za-z'
printf '%s\n' "abz ABZ" | tr 'a-zA-Z' 'b-zA-B-ZA'
```

Run, these print `Uryyb`, `mIxEd CASE 42`, and `bca BCA` — pipe the same input through your program and `diff` the two. For reversal-class output, `rev` is the oracle: `printf '%s\n' "hello" | rev` prints `olleh`.

For the function exercises, use the `harness-main` pattern from *Testing yourself like the Moulinette* does — and delete the `harness` before you submit, per the *allowed-functions* rule in *How the*

Moulinette grades; a leftover `main` collides with the grader's own and fails the build. Speed target: these are still meant to be fast — hold yourself to the level 0-1 bar in Preparation plans.

Level 2 — Reimplementing `libc`, `atoi`, bits

Level 2 (L2) widens the game: deeper `libc` reimplementation on one side, your first parsing and bit work on the other. From here on, invisible details — signedness, `INT_MIN`, a missing `+1` — decide as many grades as the logic does.

Objectives

- You can convert a numeric string to a value with sign and stop rules you can defend.
- You can explain why the most negative `int` breaks naive negation, and pick a strategy for it.
- You can compare and index bytes as `unsigned char`, and say when the cast matters.
- You can read, move, and combine single bits of a byte with shifts and masks.
- You can size, check, and fill a heap copy of a string, and free it in your own test `main` — leak-free by construction.

Concepts

Signedness of `char`. Plain `char` may be signed, and bytes beyond 127 then read as negative numbers. This demo makes it concrete:

```
#include <stdio.h>

int main(void)
{
    char    c;

    c = (char)200;
    printf("%d\n", c);
    return (0);
}
```

Compiled with `cc -Wall -Wextra -Werror` and run on an x86-64 machine, where plain `char` is signed, it prints `-56`: the byte you stored as 200, read back through the signed lens. Where plain `char` is unsigned — ARM Linux, for one — the same program prints `200`; the standard leaves that choice to the platform, which is why portable code never depends on it. Read it through the other lens — `printf("%d\n", (unsigned char)c);` — and the very same byte prints `200`. L2 is where this stops being trivia — comparison functions must compare byte values, and any code that uses a byte as an array index must never index with a negative number. The cast to `unsigned char` is the one-token fix for both.

Digit accumulation. A numeric string becomes a value one digit at a time: the value so far times ten, plus the incoming digit's value. Sign is settled before the first digit, and the loop stops at the first byte that is not a digit. That is the whole engine of `ft_atoi`; the reverse direction — peeling a value into digits — gets its full treatment in the level 3 chapter. The hazard to respect today: the most negative `int` has no positive twin, so "build positive, negate at the end" overflows on exactly one input. Stay in `int` and work on the negative side, whose range is the larger one: accumulate

the value as a negative number and flip it only when the result is positive. Widening to `long` is not the escape it looks like: the C standard lets `long` be exactly as narrow as `int` (it is, on 32-bit Linux and on 64-bit Windows); only `long long` is guaranteed at least 64 bits, and no subject that parses into an `int` needs it.

n-bounded semantics. The bounded string functions from the string and comparison days (C02–C03) are contracts about what happens at the bound, not just before it. A bound of zero means compare or copy nothing and succeed at it. Some bounded copies do not terminate what they produce — `strncpy` is the man-page classic — and some length-style functions report what they tried to build, not what fit: `strlen` returns the full length it tried to create, however little of it fit. The man page is the specification — read it like a lawyer, then implement exactly that.

Bits. A byte is eight bits. Shifting moves them: left shifts climb toward the most significant bit, right shifts fall toward the least. Masking selects them: AND with a mask keeps only chosen bits, OR merges results. `(n >> 3) & 1` reads bit 3 — that one expression, varied, is most of the level's bit work. Bit thinking is mostly new at this level; the worked example below puts the probe and the shift to work on a real byte, one bit at a time.

First heap. The level's duplicate-a-string exercise is your first exam `malloc`: size it as length plus one for the terminator, check the result against NULL before writing, and make sure whoever gets the pointer can free it. The malloc days (C07) taught the shape; here it is graded.

What the level asks

L2 deepens the libc round: `ft_strcmp` compares byte pairs as `unsigned char`; the man page promises only the sign, but a grader's `main` can print the `int` you return, so matching the real `strcmp`'s value is cheap insurance — check yours against `strcmp` on strings read from `argv` (a call on two string literals can be folded at compile time into a bare `-1`, `0` or `1`); `ft_strdup` duplicates onto the heap; `ft_strrev` reverses in place and hands its argument back.

The parser corner is where the level's difficulty concentrates. `ft_atoi` is digit accumulation with the whitespace and sign rules its subject defines; `do_op` parses two numbers and an operator and dispatches the arithmetic; what it does on a zero divisor or modulus is its subject's call — the Piscine's C11 `do-op` prints a fixed `Stop : message` for each, while a subject that promises the result fits in an `int` has promised the zero divisor away — so invent no message for it. `max` finds the largest element of an `int` array, with the empty-array answer coming from the subject.

The set-and-sequence family works on two strings at once. `inter` and `union` emit characters by membership: one keeps the characters of the first string that also appear in the second, the other merges the two strings' characters — classically each character once, in first-appearance order, with the exact dedup rules coming from the subject's examples. A seen-table — an array of 256 flags, indexed by byte value, marking which bytes have been seen or emitted — answers both "is it in the other string" and "have I emitted it already". `wdmatch` checks whether one string hides inside another in order; `last_word` mirrors the previous level's word scan from the far end. The transform `alpha_mirror` (mirror within the alphabet) is a band test with a twist.

Then the bits: `print_bits` emits a byte's eight bits most significant first, `reverse_bits` reorders them end for end, `swap_bits` exchanges the two halves, and `is_power_of_2` is a number property you can answer arithmetically or with one bit trick. All four reward the same toolkit: shift, mask, combine.

Worked example: count_trailing_zeros

An invented function, not in the pool. Spec: return how many zero bits sit at the bottom of the byte, below its lowest set bit; the all-zero byte has no set bit, so define its answer as 8. Shown with its local test harness (which never gets submitted):

```
#include <stdio.h>

int count_trailing_zeros(unsigned char octet)
{
    int count;

    count = 0;
    while (count < 8 && (octet & 1) == 0)
    {
        octet >>= 1;
        count++;
    }
    return (count);
}

int main(void)
{
    printf("%d\n", count_trailing_zeros(8));
    printf("%d\n", count_trailing_zeros(160));
    printf("%d\n", count_trailing_zeros(7));
    printf("%d\n", count_trailing_zeros(0));
    return (0);
}
```

Compiled with `cc -Wall -Wextra -Werror` and run, it prints 3, 5, 0, 8, one per line. Check them against the spec: 8 is 00001000 — three zeros below the set bit. 160 is 10100000 — the lowest set bit is bit 5, and the bits above it do not matter, so 5. 7 is 00000111 — bit 0 is already set, nothing sits below it, so 0. And 0 has no set bit anywhere, so the defined answer, 8.

Line by line. `octet & 1` is the probe: AND with the mask 1 keeps only the bottom bit and zeroes the other seven, so the whole expression is that one bit's value — 0 or 1. `octet >>= 1` is the walk: it shifts every bit one position down, so the next-higher bit becomes the new bottom bit, ready for the same probe on the next pass. Probe the bottom, shift the byte down, repeat — that pair is how you visit a byte's bits one at a time. The loop continues while the probed bit is 0, counting each one, and stops the moment a set bit reaches the bottom — `count` is then exactly the number of zeros that went by. The `count < 8` bound handles the all-zero byte without a special case: 0 shifted down is still 0, so the probe alone would never stop the loop — after eight probes the bound ends it with `count` at 8, which is precisely the answer the spec defines. The guard is built into the bound, the same move the level 0 example made with `argc`. Last, the parameter is `unsigned char` on purpose: right-shifting a negative signed value is implementation-defined — gcc and clang drag copies of the sign bit in from the top. This loop's eight-probe bound happens to stop before those copies reach the bottom, but a walk that shifts until the byte is empty never ends on a negative value; an `unsigned char` never holds one, so its shift is defined to pull in clean zeros.

Now go do the real ones — each demands something this example did not need. `print_bits` walks in the other direction: most significant bit first, all eight positions every time, each one emitted as a character — you must go get the top bits rather than wait for them to arrive at the bottom.

`reverse_bits` keeps what it reads: as it visits bits it assembles a new byte in an accumulator, so the skill is building a result across a loop, not just observing. `swap_bits` has no loop at all — it moves

both halves of the byte in a single expression, a different discipline: hold the whole picture at once instead of iterating. And `is_power_of_2` is not a walk but a property, with a one-line arithmetic answer and a one-line bit answer — finding an identity that decides it is the exercise.

Walkthroughs

The `ft_atoi` pipeline:

1. Skip the whitespace set the subject names — the set is part of the spec, not folklore.
2. Read sign characters per the subject's rule; settle the final sign before any digit.
3. Accumulate: value so far times ten, plus the incoming digit.
4. Stop silently at the first byte that is not a digit.
5. Named pitfall: the most negative int cannot be built positive and then negated. Accumulate on the negative side, whose range is the larger one, and flip for positives — decide before you write the loop, not after the overflow.

`print_bits` -class reasoning, most significant first:

1. The output is exactly eight characters, most significant bit first — every position, leading zeros included.
2. The worked example's probe only ever sees the bottom bit, and bits arrive there lowest first. Decide how you will read the top bit before the others — that decision is the exercise.
3. Each bit leaves as the character '0' or '1', not as the number 0 or 1.
4. Named pitfall: the classic subject wants the eight characters and nothing else — no trailing newline out of habit.

Pitfall and edge-case matrix

Input or situation	Classic bug	Defense
-2147483648 as input	Building the magnitude as a positive <code>int</code> first — overflow before the negation	Accumulate on the negative side and flip for positives — decided before writing the loop, not after the overflow
Bytes beyond 127 in compare- or table-driven code	Signed <code>char</code> reads them negative: wrong compare sign, out-of-bounds table index	Compare and index through an <code>unsigned char</code> cast, always
<code>do_op</code> -class / 0 or % 0	Undefined behavior — on x86-64 the program dies with a floating-point exception, exercise zeroed; <code>INT_MIN</code> with -1 is undefined and traps the same way	Test the right-hand operand before dividing, and handle it exactly as the subject says (C11's <code>do-op</code> names its <code>Stop</code> : messages); <code>INT_MIN % -1</code> traps even where the subject promises the result fits
<code>print_bits</code> -class output	A trailing newline added out of habit	The classic subject wants the bits alone; match its example to the byte
Duplicate-onto-heap sizing	Allocating length without the plus-one; the terminator lands out of bounds	Length plus one, then a <code>NULL</code> check before any write

Input or situation	Classic bug	Defense
<code>inter / union - class</code> ordering	Guessing the emission order and dedup rule	First-appearance order and dedup come from the subject's examples; a seen-table over byte values enforces both
Compare-function convention	Sign flipped because the arguments' roles got swapped mid-thought	The result's sign follows the first differing byte pair, compared as <code>unsigned char</code> ; test both argument orders

Self-check

Q1. Why can't you compute the absolute value of the most negative int the obvious way?

Answer

Its magnitude is one larger than the largest positive int, so negating it overflows — undefined behavior, and in practice a wrong value. Stay in `int`: work on the negative side, whose range is the larger one, or deal with that one value before any negation.

Q2. Your compare function disagrees with the reference exactly on strings containing accented or high-value bytes. Why?

Answer

Plain `char` may be signed, so bytes beyond 127 read as negative and the difference comes out with the wrong sign. The contract compares byte values — cast both bytes to `unsigned char` before comparing.

Q3. In a two-numbers-and-an-operator program, what must you check before evaluating?

Answer

Whether the operation divides — division or modulo — and whether the right-hand operand is zero — and how to report it, when the subject says (C11's do-op names its `Stop`: messages). That case is undefined behavior in C — on x86-64 the program dies with a floating-point exception — and a crash is a zeroed exercise. So is `INT_MIN` divided by, or taken modulo, `-1`: undefined, and it traps the same way. A subject that promises the result fits in an `int` rules out the division, but not the modulo, whose result would be 0.

Q4. A bounded compare is called with a bound of zero. What should it do?

Answer

Compare nothing and report equality — zero bytes were examined, so no difference exists. Bounded functions must do nothing gracefully; the bound is part of the contract, including at zero.

Q5. Why does duplicating a string of length ten need eleven bytes?

Answer

Length counts the bytes before the terminator, and the copy needs its own terminator too. Allocate length plus one; writing the terminator into a length-sized buffer is a heap overflow — a sanitizer or valgrind reports it at once, but a plain run often hides it, because malloc usually hands back a slightly larger block, so a passing run proves nothing.

Q6. What two output details most often zero a bits exercise whose logic is correct?

Answer

Printing a trailing newline the subject never asked for, and emitting bits least significant first instead of most significant first. Both are byte-exactness failures, not logic failures — `cat -e` against the subject's example catches both.

Verify your work

For number parsing, `expr` is an oracle for clean inputs — a harness `main` (see *Testing yourself like the Moulinette* does) prints your function's result, and the loop compares:

```
for n in 0 42 -13 2147483647 -2147483648; do
  mine=$(./yourtest "$n"); ref=$(expr "$n" + 0)
  [ "$mine" = "$ref" ] || echo "MISMATCH on $n: mine=$mine ref=$ref"
done
```

Silence means agreement. One caution: `expr " 42" + 0` refuses with `expr: non-integer argument` — `expr` does not skip whitespace, so the whitespace-and-sign cases that make `ft_atoi` interesting must come from your own hand-checked list, not the oracle.

For bit values, `bc` converts: `printf 'obase=2\n%d\n' 85 | bc` prints `1010101` — note it drops leading zeros, so compare values, not padded strings. For heap-returning work, sweep with `valgrind` exactly as the `valgrind` section prescribes:

```
for arg in "" "abc" "a much longer argument string"; do
  valgrind -q --error-exitcode=1 --leak-check=full ./yourtest "$arg" > /dev/null \
  || echo "LEAK or ERROR on: [$arg]"
done
```

The Moulinette reportedly also checks for leaks — write leak-free code regardless; it is cheap insurance. ⚠ Then hold the line from *Testing yourself like the Moulinette* does: test locally, submit once. Pace yourself against the level 2–3 bar in Preparation plans.

Level 3 — Number theory, recursion, bases, malloc'd arrays

Level 3 (L3) is where the exam stops asking you to move characters around and starts asking you to compute. The fifteen classic exercises at this level mix arithmetic, recursion, base conversion, your first malloc-returning arrays, and your first linked-list contact. The maths is never deep; validation branches and exact output decide these exercises, so precision here is worth more than speed.

Objectives

- You can peel a number into digits with `/` and `%` and print them in the correct order.
- You can write a recursive function with a base case you can state out loud before you type it.
- You can test primality with trial division up to the square root, and compute a gcd and an overflow-safe lcm.

- You can validate a base string and convert between text and numbers in any base your subject allows.
- You can malloc an array of exactly the right size, check the result, fill it, and hand it to the caller — who owns it and frees it.

Concepts

Digit peeling. $n \% 10$ is the last decimal digit; $n / 10$ drops it. For a negative n , $/$ truncates toward zero and $\%$ takes n 's sign, so the "digit" comes out negated ($-7 \% 10$ is -7) — worth knowing, since this guide's advice for the most negative `int` is to work on the negative side. The loop hands you digits backwards — least significant first. Two standard fixes: recurse on $n / 10$ before printing the current digit, or fill a buffer from its end. You met this on the first output days (C00) and the conversion days (C04); at L3 it stops being optional.

Recursion. A recursive function is a base case plus a smaller version of the same problem — say the base case in words before you type it. Recursion shines when the natural order of work is "handle the rest first, then me", which is exactly the digit-order problem above. Backtracking — try a move, recurse, undo — is the heavyweight cousin; if you did the weekend puzzle-solver project, you already trained it with explicit state, and the skill resurfaces in the exam's upper levels.

Number-theory toolbox. From the maths-and-recursion days (C05). The one algorithmic fact the grader punishes you for not knowing: a divisor of n above its square root pairs with one below, so trial division can stop once the divisor passes the square root — on a large prime, that is thousands of iterations instead of millions. Mind how you write that bound: squaring the divisor to compare it against n overflows `int` for an n near `INT_MAX` — undefined behavior, on exactly the large primes the bound exists for — so state the same limit without forming the product. For gcd, Euclid's algorithm: replace the pair with (second, remainder) until the second is zero. For lcm, divide by the gcd before multiplying, or the product can leave the type's range before the division brings it back — undefined for a signed type, a silent wrap for an unsigned one — and settle the zero inputs the subject defines before any division, since the gcd of 0 and 0 is 0.

Bases and validation. String-to-number in base B is one loop: multiply the accumulator by B , add the new digit's value — and a digit's value is its index in the base string, which makes every base the same problem, however many symbols it has. Classic validation for a caller-supplied base: at least two symbols, none repeated, no sign characters or whitespace among them (they would collide with the number's own syntax) — but your subject states its own rules; read them.

Malloc'd arrays. The heap entered the pool with level 2's string duplicate; here it grows to arrays. The shape to internalize, from the malloc days (C07):

```
int *tab;

tab = malloc(sizeof(int) * n);
if (tab == NULL)
    return (NULL);
/* fill tab[0] .. tab[n - 1], then hand it to the caller */
```

Three habits ride along: count the cells before allocating, check for `NULL` every time, and know who owns the memory — a function returning a malloc'd array transfers ownership to the caller,

who frees it. On any early-exit path, free what you already allocated — where the subject's allowed functions include `free`.

Headers and structs. Some subjects hand you a ready-made header to include but not turn in; others print the node struct and name the header it must live in, and then that header is yours to write and submit — the list days (C12) put it in `ft_list.h` and turn it in with every exercise whose file list names it (all but `ex08`, where the grader uses its own), and the classic `ft_list_size` works the same way. Copy what you are given exactly, and submit exactly the files the subject names — nothing extra, nothing missing. A header wraps its contents in an include guard so double inclusion is harmless; the Norm names the guard after the file, so `ft_list.h` gets `FT_LIST_H`:

```
#ifndef FT_LIST_H
# define FT_LIST_H
/* types and prototypes */
#endif
```

The node type behind every list exercise is a struct from the structs days (C08), and the arrow operator is the whole trick: `node->next` means `(*node).next` — follow the pointer, then take the field. The classic node:

```
typedef struct s_list
{
    struct s_list  *next;
    void           *data;
}  t_list;
```

What the level asks

The arithmetic exercises compute from numbers: `add_prime_sum` (sum the primes up to a bound), `pgcd` (greatest common divisor), `tab_mult` (a small multiplication table), and `paramsum` (how many arguments arrived) are programs that read their arguments and print the result, while `lcm` (least common multiple) is classically a function over two `unsigned int`s that returns it — the subject says which, so check before you write a `main`. Points are lost on validation branches and output format, not maths — reproduce the subject's examples byte for byte.

The string transforms now carry stateful rules: `epur_str` and `expand_str` re-space the words of their argument — classically exactly one space between words in the first and exactly three in the second, but the count is the subject's to define, so read yours. `str_capitalizer` and `rstr_capitalizer` re-case words, capitalizing the first or the last letter of each; the trap is defining "word boundary" exactly as the subject does. `hiddenp` asks whether the first argument appears in order inside the second — a two-cursor scan, no allocation needed.

The base pair: `print_hex` turns a decimal argument into lowercase hexadecimal, and `ft_atoi_base` converts a string under a caller-supplied base — but check how the base arrives: the Piscine's C04 version hands you a base string to validate and index, while the classic exam version passes the base as an `int` of at most 16 over fixed digits (`0-9`, then `a-f` in either case), with its own sign rule. Either way it is the index machine from the concepts above, built to the prototype you were given. The malloc pair: `ft_range` and `ft_rrange` return freshly allocated arrays covering the span between two bounds, one running from the first bound to the second, the other from the second back to the first — sizing, direction, and the `NULL` check are the whole exercise. Finally

`ft_list_size` is your first list exercise: walk the nodes and count them, using the node type the subject provides.

Worked example: digital root

An invented exercise — not in the pool, and that is the point: drill the mechanics here, earn the real ones yourself. The task: a program takes one argument made only of digits and prints its digital root — sum the digits, then the digits of the result, until one digit remains. No argument: print only a newline. Empty or non-digit argument: print `Error`.

```

#include <unistd.h>

int digit_sum(int n)
{
    int sum;

    sum = 0;
    while (n > 0)
    {
        sum = sum + (n % 10);
        n = n / 10;
    }
    return (sum);
}

int digital_root(int n)
{
    if (n < 10)
        return (n);
    return (digital_root(digit_sum(n)));
}

int main(int argc, char **argv)
{
    int    i;
    int    sum;
    char    digit;

    if (argc != 2)
    {
        write(1, "\n", 1);
        return (0);
    }
    i = 0;
    sum = 0;
    while (argv[1][i] != '\0')
    {
        if (argv[1][i] < '0' || argv[1][i] > '9')
        {
            write(1, "Error\n", 6);
            return (1);
        }
        sum = sum + (argv[1][i] - '0');
        i++;
    }
    if (i == 0)
    {
        write(1, "Error\n", 6);
        return (1);
    }
    digit = (char)('0' + digital_root(sum));
    write(1, &digit, 1);
    write(1, "\n", 1);
    return (0);
}

```

Compiled with `cc -Wall -Wextra -Werror` (clean under both gcc and clang), it behaves like this — real runs:

```

$ ./digital_root 9875 | cat -e
2$
$ ./digital_root 42 | cat -e
6$
$ ./digital_root 0 | cat -e
0$
$ ./digital_root | cat -e
$
$ ./digital_root 12a | cat -e
Error$
$ ./digital_root "" | cat -e
Error$

```

9875 gives $9+8+7+5 = 29$, then $2+9 = 11$, then $1+1 = 2$ — and the program prints 2 with a newline, which `cat -e` shows as `2$`.

Walk through it top to bottom. `digit_sum` is pure digit peeling: `n % 10` reads the last digit, `n / 10` discards it — and since addition ignores order, the backwards delivery is harmless here.

`digital_root` is the two-line recursion pattern: base case first (a one-digit number is its own answer), then the same problem on a smaller number; `digit_sum` shrinks, `digital_root` decides when to stop. `main` does the program-exercise chores in the standard order: the `argc` guard before anything touches `argv[1]`, then one validation-plus-accumulation pass where each character is checked against the digit band before it is used — check-then-use is what keeps garbage in the `Error` branch instead of in your arithmetic. The `i == 0` test catches the empty string, which the loop accepts vacuously (no bad characters — but no digits either). Two details worth stealing: the sum builds from `argv[1][i] - '0'` directly, so no full string-to-number conversion is needed; and the final digit prints by adding it to `'0'` and writing that one `char` — `write` wants an address and a count, so the value lands in a variable first.

Now go do the real ones. This example hands you the peeling-and-printing discipline of `print_hex` and `tab_mult`, and the validate-compute-print shape of `add_prime_sum`. Write those three cold, test with `cat -e`, and this level is mostly yours.

Walkthroughs

The `atoi-base` machine. Whiteboard steps for the `ft_atoi_base` class — the reasoning, not the code:

1. Validate the base first. For a base string: at least two symbols, none repeated, no sign characters or whitespace. For an int base: the range the subject allows. An invalid base short-circuits to the subject's failure value.
2. Skip whitespace and read signs exactly as the subject says. The Piscine's C04 version behaves "exactly like `ft_atoi`": whitespace as `isspace(3)` defines it, then any run of '+' and '-'. The classic exam version reads a '-' only as the very first character; yours decides.
3. For each character, find its value – its position in the base string, or its digit value for an int base (case rules per subject); not found means the number ends there.
4. Accumulate: multiply what you have by the base, add the new digit's value.
5. Apply the sign last. Named pitfall: the most negative int cannot be built positive and then negated – accumulate on the negative side, whose range is the larger one, and flip only for positives.

Range sizing. For the `ft_range` class:

1. Compute the cell count before touching malloc: the distance between the bounds, plus one when both ends are included, as in the classic exam pair; the C07 `ft_range` excludes max – confirm in yours.
2. Allocate that many int-sized cells and check for NULL.
3. Fill from the first bound toward the second. Whether reversed bounds are filled at all is the subject's call – the C07 `ft_range` returns NULL for them, the classic exam pair fills them. Named pitfall: hardcoding the ascending case, then looping forever on reversed bounds your subject does want filled.
4. Return the array; freeing is the caller's job.

Pitfall and edge-case matrix

These are the level-specific traps; the universal ones live in Appendix B.

Input / situation	Classic bug	Defense
Primality test on a large prime	Trial division runs to <code>n</code> ; the program looks hung	Stop at the square root — and state that bound without squaring the divisor, which overflows <code>int</code> for an <code>n</code> near <code>INT_MAX</code>
Lcm of two large numbers	<code>a * b</code> overflows before the division	Settle the zero inputs the subject defines first (the gcd of 0 and 0 is 0), then divide one operand by the gcd and multiply
Input <code>1</code> in factor-flavored tasks	Loops start at 2 and print nothing	Special-case what the subject says <code>1</code> produces (previewed here; it bites in the next level's <code>fprime</code>)
Range bounds reversed, e.g. from 0 down to <code>-3</code>	Size computed negative, or a fill loop that never terminates	Read what the subject wants first — the C07 <code>ft_range</code> returns <code>NULL</code> when min is not below max; the classic exam pair fills reversed bounds — then size and step to match
<code>epur_str</code> / <code>expand_str</code> spacing	Hardcoding the space count from memory	The classic subjects say exactly one and exactly three — but the subject in front of you defines it; read it
The most negative <code>int</code> in base work	Negating it overflows; output corrupts	Stay in <code>int</code> : work on the negative side, or deal with that one value before any negation

Input / situation

malloc fails halfway through a function

Classic bug

Early `return` leaks the earlier allocation

Defense

Free everything already allocated on every failure path — where the subject allows `free`

Self-check

Q1. Why is it enough to test divisors only up to the square root of n ?

Answer

If n has a divisor larger than its square root, the paired divisor n divided by it is smaller than the square root — so it would already have been found. No pair can hide entirely above the root.

Q2. Your range function is perfect for ascending bounds but hangs when the first bound is larger. What happened?

Answer

The fill step is hardcoded to move upward, so the walker moves away from the target bound and never reaches it. First check what your subject wants for reversed bounds — the C07 `ft_range` returns `NULL` when `min` is not below `max`, while the classic exam pair fills them — and if they are to be filled, size from the absolute difference plus one and choose the step's direction by comparing the bounds first.

Q3. As a base string, `01` is fine but `0+1` and `aa` must be rejected. Why those rules?

Answer

A sign character or whitespace inside the base would be indistinguishable from the number's own sign and padding, and a repeated symbol makes a digit's value ambiguous — its index in the base string is no longer unique. Length two is the minimum for positional meaning.

Q4. State Euclid's gcd algorithm in one sentence, including its stopping condition.

Answer

Replace the pair of numbers with the second number and the remainder of dividing the first by the second, and when the second becomes zero, the first is the gcd.

Q5. Digit peeling delivers digits backwards. What are the two standard fixes, and when do you not need either?

Answer

Recurse on the shrunk number before printing the current digit, or write digits into a buffer from its end. You need neither when order doesn't matter — summing digits, counting them — as in the worked example.

Q6. A function of yours returns a malloc'd array. Who calls free, and what is your function's own memory duty?

Answer

The caller frees the returned array — ownership transfers with the return. Your duty is the failure paths: check malloc's result, and if you exit early for any reason, free whatever you had already allocated — where the subject's allowed functions include `free`.

Q7. What does the arrow in `node->next` actually do, and when do you need it?

Answer

It dereferences the pointer and then selects the field — follow the pointer to the struct, take `next` from it. You need it whenever you hold a pointer to a struct rather than the struct itself, which is always the case while walking a list.

Verify your work

Reuse the Part II tools on this level's shapes; the target to grow toward is the under-25-minutes bar from the preparation plan.

Cross-check primality and factoring against `factor`, present on the machine and offline:

```
factor 804577
factor 42 10
```

prints `804577: 804577` — a prime, its only factor is itself — and factorizations `42: 2 3 7` and `10: 2 5`, from which you can eyeball a gcd (shared factor 2) to sanity-check your Euclid.

For hex output, mirror the grader with a diff against `printf` (see Testing yourself like the Moulinette does):

```
./print_hex 255 > got.txt
printf '%x\n' 255 > want.txt
diff -U 3 got.txt want.txt | cat -e

for n in 0 1 9 10 255 4096 2147483647; do
    ./print_hex "$n" > got.txt
    printf '%x\n' "$n" > want.txt
    diff got.txt want.txt > /dev/null || echo "K0: $n"
done
```

Against a correct implementation this prints nothing — silence is a pass; every mismatch prints a `K0` line.

For the malloc-returning exercises, wrap your function in the Part II harness `main`, print each cell, and run it under the invocation from the `valgrind` section with both normal and reversed bounds. Delete the harness before you submit — the Moulinette links function exercises against its own `main`, and a leftover one breaks the build.

Level 4 — Heap arrays, sorting, lists, 2-D thinking

The weekly exams reportedly cap around level 5, one step past this chapter; the final continues deeper. ⚠️ The level concentrates the three skills the Piscine was quietly building all along: two-level allocation, in-place mutation, and pointer surgery on linked lists. The classic slice holds nine exercises.

Objectives

- You can plan a two-level allocation — an outer array of pointers plus per-item buffers — before writing a line.
- You can compute an allocation's size in a first pass and fill it exactly in a second.
- You can read a comparator's convention off the subject — `strcmp`'s sign, or the in-order truth value a subject may define instead — and say what a graded sort must preserve: order, duplicates, and nothing extra printed.
- You can declare, pass, and call a function pointer without looking up the syntax.
- You can mutate a list through a double pointer, including removing its head, without touching freed memory.

Concepts

Arrays and in-place sorting. A bubble or selection sort is entirely enough at exam scale; what is graded is behavior — ascending order, duplicates preserved, nothing printed that the subject didn't ask for. The convention most of C shares — `strcmp`, and the Piscine's comparators in C11 and C12 — is that a comparator returns negative when the first argument sorts before the second, zero when they tie, positive otherwise: the sign of the first difference. It is a convention, not a law: a subject may define its own, and the classic exam `sort_list` does — its `cmp` returns non-zero when the two values are already in the right order and 0 otherwise, a truth value rather than a sign. Read the convention off the subject; every "is this pair out of order?" test is then one check.

Two-level allocation. A `char **` result is two layers of malloc: one outer array of pointers, then one buffer per item. Two rules make it safe. First, the outer array gets one extra cell set to `NULL` — the sentinel that tells every consumer where the array ends. Second, size before you fill: a counting pass computes exactly what to allocate, a filling pass writes exactly what was counted, and the two passes must apply identical logic or you write past your buffer. Intervals are where the off-by-ones live — know whether your bounds are half-open (end excluded, like the range exercise from the malloc days) or inclusive; the subject's examples settle it.

Function pointers. From the function-pointer day (C11). The declaration reads inside-out — `int (*f)(int, int)` is "f is a pointer to a function taking two ints, returning int" — and using one is anticlimactic:

```
int (*f)(int, int);
int r;

f = cmp_asc;          /* f now points to cmp_asc */
r = f(3, 7);          /* call through the pointer */
```

With `cmp_asc` returning `-1`, `0` or `1` as its first argument is smaller than, equal to, or larger than its second, `f(3, 7)` returns `-1` — compared, not subtracted: first-minus-second overflows when the operands are far apart, `INT_MAX` and `-1` for instance. Subjects at this level hand your function a comparator or an applier; your job is only to call it at the right moment and interpret its result by the convention the subject states.

Linked lists, full treatment. The node is the L3 struct; the new material is mutation. Three laws cover every list exercise from the list days (C12). One: to change where the list starts, the change must reach the caller — through the address of the head pointer, a `t_list **`, or by returning the

new head; the subject's prototype decides which (`ft_list_remove_if` takes the address, the classic `sort_list` returns the head). Two: never read from a node after freeing it; save `node->next` into a temporary before the free. Three: wiring order — when inserting, point the new node at its successor before anyone points at the new node, so the chain is never broken. Traversal itself stays the L3 cursor loop; mutation is where the segfaults come from.

What the level asks

Two exercises are pure allocation-sizing discipline: `ft_split` cuts a string into a freshly allocated NULL-terminated array of words — the two-level case — and `ft_itoa` turns an `int` into a freshly allocated string — a single buffer, whose whole difficulty is exact sizing (digit count, plus sign, plus terminator) and the most negative `int`. `fprime` prints the prime factorization of its argument in a precise output format; the maths is L3 trial division, the risk is the format and the special small inputs.

Two are word-order transforms that need no allocation if you think in cursors: `rev_wstr` prints the words of its argument in reverse order, and `rostring` moves the first word to the end. Both are separator-scanning problems in the `first_word` lineage, graded on exact spacing.

Sorting arrives twice: `sort_int_tab` sorts an `int` array in place, and `sort_list` sorts a linked list using a comparator you are handed. The list versions of iteration arrive with `ft_list_foreach` (apply a function to every node's data) and `ft_list_remove_if` (delete every node whose data matches a reference — the double-pointer exercise).

One attested addition belongs to this level's orbit: `flood_fill`, region-filling on a grid, reported at level 4 by several sources and level 5 by another — a contested slot from outside the classic list, so treat its placement as campus-dependent. 🏠 Grid representation and traversal are covered in the next chapter and the far-side chapter; the competency itself is the weekend grid work resurfacing.

Worked example: run-length encoder

Invented task, real discipline: write a **function** that returns a freshly allocated run-length encoding of its input — each run of a repeated character becomes the character followed by the run's length as a single digit, so "aaabcc" encodes to "a3b1c2"; a run longer than nine is written as several runs of at most nine. This is the two-pass allocation discipline — measure, allocate once, fill — that `ft_split` and `ft_itoa` both demand, on a task that is neither.

```

#include <stdlib.h>
#include <unistd.h>

int encoded_size(char *s)
{
    int size;
    int run;
    int i;

    size = 0;
    i = 0;
    while (s[i] != '\0')
    {
        run = 1;
        while (run < 9 && s[i + run] == s[i])
            run++;
        size = size + 2;
        i = i + run;
    }
    return (size);
}

char *rle_encode(char *s)
{
    char *out;
    int i;
    int pos;
    int run;

    out = malloc(encoded_size(s) + 1);
    if (out == NULL)
        return (NULL);
    i = 0;
    pos = 0;
    while (s[i] != '\0')
    {
        run = 1;
        while (run < 9 && s[i + run] == s[i])
            run++;
        out[pos] = s[i];
        out[pos + 1] = (char)('0' + run);
        pos = pos + 2;
        i = i + run;
    }
    out[pos] = '\0';
    return (out);
}

int main(int argc, char **argv)
{
    char *encoded;

    if (argc != 2)
        return (1);
    encoded = rle_encode(argv[1]);
    if (encoded == NULL)
        return (1);
    write(1, "[", 1);
    write(1, encoded, encoded_size(argv[1]));
    write(1, "]\n", 2);
    free(encoded);
}

```

```

    return (0);
}

```

It compiles clean with `cc -Wall -Wextra -Werror` under both `gcc` and `clang`. Run it — brackets added by the harness to make every byte visible:

```

$ ./rle_demo "aaabcc" | cat -e
[a3b1c2]$
$ ./rle_demo "aaaaaaaaaab" | cat -e
[a9a3b1]$
$ ./rle_demo "" | cat -e
[]$
$ ./rle_demo "x" | cat -e
[x1]$

```

And the memory verdict under `valgrind --leak-check=full`: All heap blocks were freed -- no leaks are possible, ERROR SUMMARY: 0 errors from 0 contexts.

Walk it through. `encoded_size` is pass one: it walks the string run by run — `run` grows while the next character repeats the current one, up to the spec's cap of nine — and adds two bytes per run, one for the character and one for its count. It allocates nothing; it only measures. `rle_encode` is pass two, and its loop is line-for-line the same run detection as `encoded_size` — that duplication is not laziness, it is the safety property: because both passes agree on what a run is, the fill writes exactly the bytes the measurement paid for, then places the terminator and returns. The cap is what keeps every count a single digit, so `'0' + run` is the whole conversion — value in, character out, the L1 move. The one `malloc` is checked, and the `+ 1` buys the terminator's byte. The `main` is the harness pattern from Part II: it prints exactly the bytes pass one measured, bracketed, so an empty result, a stray space, or a missing byte is visible on sight — and a fill that ever fell short of its measurement would show up under `valgrind` as unwritten bytes being printed. It frees what it received, which is how the `valgrind` run stays clean. The harness never gets submitted — for a function exercise the Moulinette supplies its own `main`, and yours would collide with it.

Now go do the real ones: `ft_split` and `ft_itoa` are this measure-then-fill discipline on payloads of their own. What each one has to measure, and how it fills what it measured, is the exercise — the worked example is the drill; those two are the exam.

Walkthroughs

The split discipline. Whiteboard only — steps, not statements:

1. Decide what a separator is. The subject decides, not memory.
2. Pass one: count the words. A word begins wherever a non-separator follows either a separator or the start of the string.
3. Allocate the outer array: one pointer per counted word, plus one for the NULL sentinel.
4. Per word: measure its length, allocate length plus one, copy its bytes, terminate it.
5. Put NULL in the last outer cell.
6. If any allocation fails midway, return what the subject specifies for failure – freeing what was already allocated only if free is among the subject's allowed functions (C07's and C09's `ft_split` allow `malloc` alone). Named pitfalls: pass one and pass two disagreeing on separators (buffer chaos), a forgotten sentinel (the consumer runs off the array), and, where free is allowed, the midway-failure leak.

Removing from a list, head included. For the `ft_list_remove_if` class:

1. Matches at the head first: while the head node itself matches, the head pointer itself must move – which is why the function receives the address of the head pointer, not a copy of it.
2. Before freeing any node, save its successor into a temporary. Freed memory is off-limits, including its next field.
3. After unlinking a node, re-examine the same position – the next node might match too. Consecutive matches are the classic killer.
4. Advance the cursor only when nothing was removed at the current position.

Pitfall and edge-case matrix

Level-specific traps; the universal rows are Appendix B's.

Input / situation	Classic bug	Defense
Split-class result consumed by the grader	Missing <code>NULL</code> sentinel on the outer array; iteration runs into unmapped memory	Allocate words plus one; set the last cell to <code>NULL</code>
Any word or number buffer	Forgetting the terminator's <code>+ 1</code> in the size	Size = payload + 1, always; recount on the subject's longest example
<code>malloc</code> fails on word three of eight	Function returns; words one and two leak	Free them first only where the subject allows <code>free</code> — C07's and C09's <code>ft_split</code> allow <code>malloc</code> alone, and there a <code>free</code> call is the bug, not the leak — then return the subject's failure value
The most negative <code>int</code> into <code>itoa</code> -class sizing	Negating overflows; length is one short for the sign	Stay in <code>int</code> : work on the negative side, whose range is the larger one, or deal with the one value that has no positive counterpart before any negation — and count the sign's byte in the size
Comparator-driven sort	Sign convention inverted; output exactly reversed	Read the convention off the subject — <code>strcmp</code> 's sign for the Piscine's comparators, a truth value (non-zero = already in order) for the classic exam <code>sort_list</code> — then test with a two-element case
Node removal	Reading <code>node->next</code> after <code>free(node)</code>	Save the successor first, free second

Input / situation	Classic bug	Defense
fprime -class input 1	Factor loop starts at 2, prints nothing	Special-case it to whatever the subject's examples show
Sorting with duplicates	Dedup logic sneaks in; grader expects every element kept	Sort must keep every element — equal values are neither dropped nor treated as out of order

Self-check

Q1. A split-class function counted `N` words. How many cells does the outer array need, and why?

Answer

`N` plus one. The extra cell holds the `NULL` sentinel, which is the only way a consumer can know where the array of pointers ends.

Q2. Why must the counting pass and the filling pass share identical separator logic?

Answer

The count decides the allocation size; the fill decides the bytes written. If they disagree — one treats a tab as a separator, the other doesn't — the fill writes more or fewer items than were paid for, and writing past the allocation corrupts the heap.

Q3. What is special about the most negative `int` in number-to-string work?

Answer

It has no positive counterpart in `int` — negating it overflows. Its text is also the longest an `int` can produce, sign included, so it stresses both the sizing and the sign handling. Stay in `int` : work in negative space, whose range is the larger one, or deal with that one value before any negation.

Q4. A comparator returns the first argument minus the second. Ascending or descending — and how would you flip it?

Answer

Ascending: a negative result means the first is smaller and already in place. Swapping the operands flips the order; negating the result does too, unless the result can be `INT_MIN`, whose negation overflows. One two-element test tells you which order you actually wrote. And distrust the subtraction itself: first-minus-second overflows when the operands are far apart (`INT_MAX` and `-1`) — undefined behavior, in practice a wrong sign — which a comparator built from `<` and `>` never does.

Q5. Why does a remove-matching-nodes function take the address of the head pointer?

Answer

Because the head itself may be removed, and then the caller's own head pointer must change. A copy of the pointer could relink internal nodes but could never move the list's entry point.

Q6. You freed a node and then read its `next` field to continue the loop. What happens?

Answer

Undefined behavior — the memory is no longer yours; it may still look right, then crash under the grader. Save the successor into a temporary before the free, and continue from the temporary.

Q7. Your split-class function receives a string made only of separators. What should it return?

Answer

A valid allocated array containing zero words: just the `NULL` sentinel in the first cell. No words is a normal result, not an error.

Verify your work

Make empty and invisible tokens visible: wrap your function in the Part II harness `main` and print every token between brackets, the way the worked example's harness does — `[tok]` exposes an empty token as `[]` and a swallowed space instantly.

Sweep allocations with `valgrind` across a hostile input list — the loop below stays silent when every case is clean:

```
for s in "" "a" " " "aaabcc"; do
    valgrind -q --leak-check=full --error-exitcode=1 ./your_harness "$s" > /dev/null \
        || echo "K0: [$s]"
done
```

The Moulinette reportedly also checks for leaks — write leak-free code regardless; it is cheap insurance. ⚠️

For `sort-class` output, print one element per line from your harness and diff against the system's `sort`:

```
./sort_harness 5 3 9 1 3 > got.txt
printf '%s\n' 5 3 9 1 3 | sort -n > want.txt
diff -U 3 got.txt want.txt | cat -e
```

Silence is a pass; note the duplicated 3 — `sort -n` keeps both, and so must you.

Level 5 — Parsers, stack machines, interpreters

Level 5 (L5) is the classic pool's summit: seven exercises where the input is a little language and your program is its machine. Nothing here needs an algorithm you don't already own — the difficulty is state: reading a symbol, updating a model, and refusing bad input without crashing. If you parsed a puzzle string or a board file during the weekend projects, you have already trained exactly this: parse into a structure, validate, act — with one error path for everything the subject calls malformed.

Objectives

- You can model a task as read-symbol, update-state, continue-or-fail.
- You can implement an array-backed stack whose every operation checks its preconditions first.

- You can route every malformed input into the one error output the subject defines, where it defines one, without leaking or crashing.
- You can treat memory as raw bytes through `unsigned char` and reason about a hex dump's layout.
- You can read from standard input in chunks, driven by the byte count `read` returns.

Concepts

Bytes, not chars. Byte-level exercises want memory reinterpreted, and the tool is one cast:

```
const unsigned char *p;

p = (const unsigned char *)addr;    /* bytes, 0..255 */
```

`void *` means "some memory"; casting to `unsigned char *` turns it into inspectable bytes. Unsigned matters: whether plain `char` is signed is the platform's choice, and on x86-64 it is — so byte values above 127 turn negative and poison comparisons and formatting. Each byte then splits into two hex digits — the high four bits and the low four bits — which is the entire alphabet of a hex dump. The layout of a classic dump (an address column, hex columns, a printable-characters column, non-printables shown as a placeholder) you have met in the byte-printing days (C02, C10); which zones appear, and their exact widths, are always the subject's.

Syscall input. When a subject wants standard input, the loop is: `read` into a buffer, process exactly the number of bytes the call returned, repeat until it returns zero — and stop on a negative return too: that is an error, not a count. The count is the only truth — binary-safe code never trusts a terminator it didn't place, and never assumes one `read` delivers everything. Write out only what you counted. This is the file-days discipline (C10) resurfacing under exam pressure.

State machines with an Error contract. The level's parser exercises share one skeleton: initialize state (a stack, a tape, a bit field, a board) — consume input one token at a time — per token, first check the preconditions, then act — and at the end, verify the final state is legal. A violated check collapses into whatever the subject says bad input produces — and only where it says so: a subject that guarantees valid input, or that calls unknown characters comments (the classic brainfuck does both), wants no error branch at all, and an `Error` it never asked for is a wrong output. Writing the checks before the actions is the entire trade secret of this level.

What the level asks

Three exercises are stack problems in different clothes. `brackets` validates that delimiters nest properly in each argument — open ones are remembered, close ones must match the most recent memory. `rpn_calc` evaluates an arithmetic expression written operator-last, with a stack of operands; malformed anything means the error output. `check_mate` swaps the stack for a board: given a chess-flavored position as input, decide whether the king is attacked — the subject defines the symbols and the attack rules, the exercise is coordinate scanning with bounds guards on a board whose size varies; several sources place this exercise at level 4 rather than 5, so its slot is campus-dependent. 🏠

`brainfuck` is the interpreter: a code string of eight one-character commands driving a data tape — a dispatch loop plus matching-bracket jumps; the subject specifies the tape size — read it. `options` parses letter flags into a 32-bit field (classically `a` is bit 0 and the letters map upward) and prints

or reacts per the subject, with an explicit branch for invalid options. `print_memory` is the hex-dump exercise: bytes of a region rendered as rows of hex beside their printable characters, where which zones appear at all — the Piscine's C02 version leads each row with an address, the classic exam version has none — and the exact widths, grouping, and padding come from the subject's example. `ft_itoa_base` closes the base saga in the allocation direction; classically the minus sign appears only in base 10, and every other base reads a negative value as unsigned — but that, too, is the subject's call.

Two attested additions orbit this level in newer sources only, outside the classic enumeration: `biggest_pal` (find the longest palindromic stretch inside a string) and `cycle_detector` (decide whether a linked structure loops back on itself). Treat both as era- and campus-dependent extras, not classic-pool members. 🏠

Worked example: a stack-toy interpreter

Invented language, real machinery. An invented subject: a program takes one argument, a program in a four-symbol language. A digit pushes its value; `d` duplicates the top; `s` swaps the top two; `p` pops the top and prints it on its own line. Anything else — an unknown symbol, an operation on too few values, a full stack, or a missing argument — prints `Error`. It drills the guard-then-act rhythm and the single error path on a language that is none of the level's exercises; which of them need a stack, and which have an error output at all, their subjects decide.

```

#include <unistd.h>

#define STACK_MAX 4096

void    print_value(int value)
{
    char    digit;

    digit = (char)('0' + value);
    write(1, &digit, 1);
    write(1, "\n", 1);
}

int run(char *code)
{
    int stack[STACK_MAX];
    int top;
    int tmp;

    top = 0;
    while (*code != '\0')
    {
        if (*code >= '0' && *code <= '9')
        {
            if (top == STACK_MAX)
                return (1);
            stack[top++] = *code - '0';
        }
        else if (*code == 'd')
        {
            if (top == 0 || top == STACK_MAX)
                return (1);
            stack[top] = stack[top - 1];
            top++;
        }
        else if (*code == 's')
        {
            if (top < 2)
                return (1);
            tmp = stack[top - 1];
            stack[top - 1] = stack[top - 2];
            stack[top - 2] = tmp;
        }
        else if (*code == 'p')
        {
            if (top == 0)
                return (1);
            print_value(stack[--top]);
        }
        else
            return (1);
        code++;
    }
    return (0);
}

int main(int argc, char **argv)
{
    if (argc != 2 || run(argv[1]) != 0)
    {
        write(1, "Error\n", 6);
        return (1);
    }
}

```

```

    }
    return (0);
}

```

It compiles clean with `cc -Wall -Wextra -Werror` under both gcc and clang, and runs like this:

```

$ ./stacktoy "12sp" | cat -e
1$
$ ./stacktoy "5dpp" | cat -e
5$
5$
$ ./stacktoy "sp" | cat -e
Error$
$ ./stacktoy "7x" | cat -e
Error$
$ ./stacktoy "5px" | cat -e
5$
Error$
$ ./stacktoy "" | cat -e
$ ./stacktoy | cat -e
Error$

```

`12sp` pushes 1 and 2, swaps them so 1 is on top, pops and prints 1. `5dpp` duplicates the 5 and prints it twice. The empty program prints nothing and succeeds — no rule was violated. And `5px` shows a policy this invented subject chose: output already produced stays, the `Error` follows it; a real subject defines its own policy, which is precisely why you read subjects.

Walk it through. `print_value` converts a value to its digit character and writes that one byte plus a newline — values here can only be 0 through 9 since only digits push and no operation creates new values. `run` owns the machine: the stack is a plain array and `top` is the state — it counts how many values are live and names the next free slot. The loop is a dispatch chain over the current symbol, and every arm follows the same two-beat rhythm: guard, then act. A digit push first checks the stack isn't full. `d` requires something to duplicate and room for the copy — two distinct preconditions, both checked. `s` requires two values. `p` requires one, and note `stack[--top]`: decrement first, then read — the value at the old top. The final `else` is the unknown-symbol trap, and this ordering matters: because every arm returns 1 the instant a precondition fails, no state is ever corrupted before the failure is noticed. `run` reports; it does not print the error. That separation lets `main` collapse every failure — wrong argument count or any runtime violation — into one `Error` line, the single-error-branch discipline that a parser subject with an error output demands.

Now go do the real ones: `rpn_calc`, `brainfuck` and `options` each keep a state of their own and change it one symbol at a time. What that state is, what each symbol does to it, and what — if anything — its subject calls an error is the exercise; the worked example is the drill, those three are the exam. Before you reach for an `Error` arm in `brainfuck`, read what its subject says about characters outside the language.

Walkthroughs

The rpn-class evaluation loop. Steps only:

1. Cut the input into tokens by the subject's separators.
2. A number token: push it.
3. An operator token: check the stack holds at least two values — pop two, apply, push the result. For division-flavored operators, check the divisor before applying; the subject says what a zero divisor produces.
4. End of input: exactly one value may remain — that is the answer. Zero values, or two or more, is the error case. Named pitfalls: leftover operands silently ignored, a single-operator input, the empty input.
5. Any unrecognized token, at any point, is the error case immediately.

Hex-dump layout reasoning. For the `print_memory` class:

1. Walk the region in fixed-size rows — classically sixteen bytes.
2. Each row has the zones the subject's example shows: the bytes in hex and the bytes as characters, led by an address only when the example has one (C02's does; the classic exam version's does not).
3. In the character zone, printables appear as themselves and everything else as the subject's placeholder.
4. The final row is usually short — the hex zone must be padded so the character zone still lines up in columns.
5. Widths, grouping, separators, and the placeholder are the subject's; reproduce its example byte for byte rather than any dump format you remember.

Pitfall and edge-case matrix

Level-specific traps; universals live in Appendix B.

Input / situation	Classic bug	Defense
More pops than pushes	Stack index goes negative; garbage reads or a crash	Guard before every pop — check emptiness first
Very long input onto a fixed stack	Push past the array's end corrupts the frame	Guard the push against the capacity too
Empty input, or a lone operator	Falls through to printing garbage or nothing	The end-state check: exactly one result value, else the error branch
Bracket-jump scanning	Jump takes the first closer, not the matching one — or runs off the string	Track nesting depth: openers raise it, closers lower it, stop at zero; hitting the string's end first is malformed input — a subject that guarantees valid code (the classic brainfuck) never feeds you that case, so it needs no branch for it
Flag parsing	Wrong letter-to-bit mapping, or invalid letters accepted	Classically <code>a</code> is bit 0 and letters map upward; every character outside the set the subject lists — mind the case — routes to its invalid-option branch
Variable-size boards	Indexing outside the board on edge coordinates	Guard every coordinate against the board's bounds before reading — grid discipline from the weekend projects
Byte formatting	Signed <code>char</code> sign-extends; bytes above 127 print wrong	Cast to <code>unsigned char</code> before any formatting or comparison

Input / situation	Classic bug	Defense
Base output with signs	A minus sign emitted in every base	Classically the sign appears only in base 10 and every other base reads a negative value as unsigned — and your subject has the final word

Self-check

Q1. Your stack code pops and only then checks whether the stack was empty. Which inputs kill it?

Answer

Any input where an operation needs more values than were pushed — including the very first token being an operator, and the empty-stack duplicate. The read at a negative index is undefined behavior. Guards come before actions, always.

Q2. An rpn-class evaluation finishes with two values on the stack. What is the correct output?

Answer

The error output. A well-formed expression consumes everything into exactly one result; leftover operands mean the expression was malformed, no matter how valid the arithmetic that did happen was.

Q3. Why does byte-dump code cast to unsigned char before formatting?

Answer

Plain `char` is signed on x86-64, so byte values above 127 become negative numbers; comparisons and hex formatting then produce nonsense. Unsigned makes every byte 0 to 255, which is what a dump means.

Q4. How do you find the close-bracket that matches a nested open-bracket?

Answer

Scan forward keeping a depth counter: raise it on every opener, lower it on every closer, and the closer that brings it to zero is the match. Reaching the end of the input first means the brackets don't balance.

Q5. A subject says to read the input from standard input. When do you stop, and what do you trust?

Answer

Stop when `read` returns zero — or a negative value, which is an error, not a byte count; what to print then is the subject's call. Trust only the returned count: process exactly that many bytes per call, never a terminator you didn't place, and never assume one call delivered the whole input.

Q6. In flag-parsing exercises, how does a letter become a bit, and what does an invalid letter do?

Answer

Classically the letter's distance from `a` is its bit position, so `a` is bit 0. Any character outside the valid set routes to the subject's explicit invalid-option branch — printing whatever the subject shows, not silently skipping.

Verify your work

Parser exercises die on adversarial input, so keep a list of nasties and fire it in one loop. Fire it at the worked example first:

```
for a in "" "p" "s" "5dp" "12spp" "9x" "5 p"; do ./stacktoy "$a" | cat -e; done
```

It prints, in order: nothing for the empty program, `Error$`, `Error$`, `5$`, `1$` then `2$`, `Error$`, and `Error$` (the space is not in the language). Reuse the idea on the real exercises: empty string, a lone operator, garbage characters, a trailing operator, and no arguments at all — every one through `cat -e`, before `grademe` ever sees it.

For dump-layout exercises, use the system's dumpers as a structural reference:

```
printf 'Hello, exam!\001\002' | hexdump -C
```

prints an address column, the hex bytes, and `|Hello, exam!..|` — the two control bytes rendered as placeholders. `xxd` shows the same data with different grouping. Compare structure, never bytes: the exact format that counts is the one in your subject's example. Against the clock, the target here is the under-45-minutes parser bar from the preparation plan.

The far side — levels 6 to 10

Past level 5, firm information runs out. Nearly everything known about the final's upper levels comes from a single first-hand account of a 2017 final at one campus, which describes, level by level: per-letter counting at level 6, ordering words at level 7, grid regions and giant-number addition at level 8, a graph-distance problem at level 9, and MD5 at level 10. ⚠️ Treat this as what the far side felt like in 2017 — not as a pool; campuses and eras differ in exactly this territory. 🏠 The final reportedly runs up to level 10, where the weekly exams stop around level 5. ⚠️

What the classes demand, concept by concept:

Counting letters at scale. A frequency table over a string, with the subject's rules for case and for the order in which counts are reported. It is tokenizing plus an array of counters — level-4 skills, more bookkeeping.

Multi-key ordering. Order words by length, breaking ties alphabetically — a two-stage comparator (compare lengths first; only on a tie compare text) driving the sorting you already own. The 2017 account describes grouped output, but only loosely; the subject defines the format.

Grid regions. Find connected regions of walkable cells on a grid of walkable and blocked cells — which characters mark which is the subject's to define, and accounts differ on it. 🏠 The competency is the weekend grid work: represent the board, guard every coordinate, and spread through neighbors with explicit state.

Arbitrary-precision addition. Numbers too big for any integer type, kept as strings: schoolbook column addition, walking both strings from their right ends with a carry, writing the result backwards. Nothing but L3 digit discipline applied to text.

Graph and tree measures. The 2017 account's level-9 problem was a graph diameter — the longest distance between two nodes. The tree-shaped idea to hold onto, from the binary-tree days (C13): a tree's height is one plus its tallest child's height, and a diameter through a node is the sum of its two tallest child-heights — two recursions you can reason about on paper.

MD5, the summit. The 2017 account's level 10 was implementing the MD5 hash, of which its author wrote: "No one, as far as I know, has gotten this right." ⚠ The top levels are effectively unreachable in-exam; they decide ranking among the strongest, not passing.

Be honest with your hours: for most readers this chapter is reading, not prep — if you are chasing a top score, the marginal points live at levels 5 to 7, and the last levels are not worth prep time. ⚠ If you do arrive here on exam day with time to spare, notice that every class above is a composition of things this guide already made you drill: tokenizing, sorting with comparators, grid guards, digit peeling, recursion with a stated base case. The far side asks for nothing new — only bigger combinations of what you already drilled.

Part IV — Strategy and preparation

Exam-day playbook

A full-length final is long enough to get tired, and tiredness — not difficulty — causes most unforced errors. So budget effort, not the clock: time spent in the exam is not scored, and you may leave whenever you want, which means staying to attempt one more level costs nothing but energy. With that in mind, here is the guide's suggested shape for the day — advice, not an exam rule:

- **Secure the base.** Spend the first stretch of the day — roughly the first three hours, if you want a number — clearing levels 0 through 3 on first attempts. Slow is fine. A clean first attempt is worth more than two rushed ones: the ladder pays 9 for the first try and only 4 for the retry.
- **Push the middle.** Give the middle of the day to L4 and L5. Allow more time per exercise than feels natural, and spend a real share of it on leak checks and boundary cases before you even think about submitting.
- **The far side, only with time to spare.** If L0-5 are behind you and hours remain, keep going — but know that one first-hand account describes the top levels as effectively unreachable in-exam; they decide ranking, not passing. ⚠

For every exercise, run the same six-step protocol in order — naming mistakes and stray files are a leading cause of avoidable zeros, and the protocol exists to make them impossible:

1. **Read the subject twice.** Function exercise or program exercise? Copy — do not retype — the exact directory name, file name, and prototype. Then read the allowed-functions header; it is absolute.
2. **Create the directory with the copied name** under `~/rendu/`, and inside it every file the subject expects — usually just `<exercise_name>.c`, though a list exercise can ask for its header too — each named exactly as the subject spells it.
3. **Function exercise? Delete your test `main` now**, before staging. The Moulinette compiles your file together with its own `main.c`, so a leftover `main` is a duplicate-symbol build failure and a 0.
4. **Clean strays, then look.** Remove `a.out`, `.o` files, and editor backups — everything you push is collected and listed in your trace. Run `git status` (or `git ls-files`) until exactly the files the subject names are going in — nothing more, nothing missing.
5. **Add, commit, push.** Grading happens server-side on what you pushed; an unpushed commit is an unsubmitted exercise.
6. **Run Appendix A's checklist, then `grademe`, confirm — and read the trace** top to bottom whatever the verdict, as taught in Reading a trace, line by line.

The shell part of the loop, ready to adapt:

```
cd ~/rendu/my_exercise
rm -f a.out *.o          # strays out before staging
git status               # only the files the subject names should be left to add
git add my_exercise.c
git commit -m "my_exercise"
git push
```

Treat grademe as a verdict, never as a debugger. Each failed attempt at a level drops the ladder from 9 points to 4, and fail twice and the level is worth 0 — the third exercise only moves you forward. On top of that, one 2021 account from 42 Abu Dhabi describes an exponential cooldown between attempts — roughly 12 minutes, then 40, then an hour and a half. ⚠️ Whether or not your campus enforces one, the discipline is the same: test locally until you cannot break your own program, then submit once.

When a 0 lands anyway, work the trace part by part (Reading a trace, line by line; Appendix D is the pocket version). Copy the exact compile line and run it on your file; re-run the failing test's exact invocation locally; read the diff with the decoder — - lines are yours, + lines are expected, \$ marks every line end. Fix the one thing the trace shows, re-verify, and take the retry: 4 points is not 9, but it clears the level and keeps you moving. Some campuses withhold traces. 🏠 If yours does, re-derive the failure from Appendix B: the classics are an unterminated string, reading `argv[1]` without an `argc` guard, and writing past a malloc'd buffer, plus the eternal missing trailing newline, since the smallest output mismatch is a 0 for the exercise.

And if the morning goes badly, do the arithmetic before you spiral: three clean first-attempt levels are about 27 points, and 25 validates the exam. One 0 is recoverable bookkeeping, not a verdict on your day — the retry pays 4, the next level pays 9 again, and the day is long.

Preparation plans

Three ways through this guide sketched three reader paths; here each one expands into a full training plan. All three use the same three gates — suggested benchmarks from community prep experience, not exam rules:

- any L0-1 exercise in under 10 minutes, cold;
- any L2-3 exercise in under 25 minutes, compiling clean under `-Werror`, leak-free;
- a working parser of the `rpn_calc` / `brainfuck` class in under 45 minutes.

Exam in 3 days. You are verifying, not learning. Day 1: Part I in full, then a skim of Appendices A and F — the rules and the map. Day 2: the Level 0 through Level 3 chapters — objectives, pitfall matrices, and self-checks — doing a handful of L0-1 pool exercises against the 10-minute gate, then the `cat -e` and diff drills from Compile exactly like the grader and Testing yourself like the Moulinette does until the loop is reflex. Day 3: the Exam-day playbook, then a timed half-day run under real conditions, then the night-before cards: Appendices A, B, and D. The half-day run needs a simulator, and simulators drift — see Practicing under real conditions. 🏠 If a gate fails, stay at that level — passing is L0-3 done cold, not L5 done shakily.

Steady 2 weeks. Read in order, about a level chapter per day and a half, and after each chapter run the retrieval loop that this whole guide is built around: read the chapter, close it, write that level's pool exercises cold, break them with the diff/fuzz/valgrind recipes from Part II, and only then reopen the archetype notes to see what you missed. The loop is the plan — the chapters exist

to make your own attempts land, not to replace them. On days 11 and 12, run one full timed simulator session; spend the last days re-drilling whichever matrix rows caught you.

Chasing the top score. Assume L0-3 are cold and stop polishing them — three clean first-attempt levels already clear the pass line. The marginal points live at L5-7; the same 2017 account that mapped the far side — echoed by the community advice ranked in Appendix H — holds that L8-10 are effectively unreachable in-exam and not worth preparation time for most. ⚠️ For calibration, that account's author scored 76/100 after reaching nine levels, in a Piscine whose top score was 81. ⚠️ So train the 45-minute parser gate until it feels roomy, go deep on the Level 5 chapter, and build debugger fluency in Part II — speed at L5 buys more than theory at L9.

Practicing under real conditions

Practice simulators for the real submission loop exist: `JCluzet/42_EXAM` — the engine behind `grademe.fr` and the most current emulator, though its own discussion threads note it exposes only the week-1 and week-2 pools — and `emreakdik/42ExamPractice`. Tools like these drift and disappear, so check what still works during your own Piscine. 🏠

However you practice, make it honest: offline, a visible timer, phone in another room, the editor you will actually have, and the submit-once rule enforced on yourself — one grading check per exercise, only after your own tests pass. Rehearse the git loop until it is muscle memory: exact directory name, a per-exercise `.gitignore` that ignores everything, itself included, except the files the subject names (pushed, it would be one more file than the subject asks for), `git status` before every push. On exam day, the C should be the only thing that needs your full attention.

If a weekly exam is closer than the final: everything here transfers, because the weeklies draw on the same pool at lower levels. The loop you rehearse now is the loop you will run in all of them.

What the rushes were really teaching

If you did a weekend rush, you already trained for the exam's hardest exercises — that is this guide's reading of the overlap, not something 42 announces. Four competencies transfer: representing a 2-D grid and walking it without stepping out of bounds; backtracking with explicit state (try, mark, recurse, unmark); parsing input into a structure with a validated `Error` branch; and producing exact-format output across many cases. The first three pay off directly in Level 5 — parsers, stack machines, interpreters, and all four in The far side — levels 6 to 10.

If you skipped the rushes, or slept through yours, three catch-up drills — all on tasks you invent yourself, none of them exam content:

1. **Grid walk.** Draw a small grid on paper, store it as an array of strings, and write a program that prints the coordinates of every neighbor of a chosen cell. Then make it survive corner cells and a one-by-one grid.
2. **Maze retreat.** Sketch a tiny maze of your own design and write an escape that marks cells visited, recurses, and unmarks on dead ends. The try-undo rhythm is the whole lesson.
3. **Strict little parser.** Invent a five-character command format — a clock time, say — and write a validator that reprints it normalized, or prints `Error` and nothing else, for every malformed input you can dream up.

None of these will appear in an exam; that is the point. They build the muscles the hard levels assume you have.

Part V — Appendices

Appendix A — Pre-submit checklist

The playbook's protocol is what you do, in order; this card is what you verify, in the thirty seconds before typing `grademe`. Ten yes-or-no checks — any "no" means stop:

- Directory and file names match the subject exactly — copied, not retyped?
- Prototype matches the subject character for character?
- Function exercise → your test `main` is deleted?
- Every library function you call appears in the subject's allowed-functions header? (Your own helper functions are not on that list, and need not be.)
- `cc -Wall -Wextra -Werror` compiles with zero warnings — add `-c` for a function exercise, whose `main` is the grader's?
- Sanitizer and `valgrind` runs are clean? (The grader reportedly also checks for leaks — leak-free is cheap insurance regardless.) ⚠
- Output checked with `cat -e` on every example — a `$` everywhere the subject shows a line end?
- Ran it with no arguments and with `" "` — no crash?
- `git status` shows exactly the files the subject names — all of them, and no `a.out`, no `.o`, no backups?
- Pushed — and the push succeeded — before `grademe`?

Appendix B — Universal edge-case matrix

These twelve rows apply at every level; each level chapter adds its own exercise-class rows on top. Before any submission, run your program — mentally at minimum, literally when you can — against every row.

Category or situation	Classic bug	Defense
Strings" (empty string)	Code assumes at least one character and reads meaning into <code>str[0]</code>	A zero-length input should fall through your loops untouched — vacuous truth is your friend
Strings " (whitespace only)	Word scanner runs past the terminator hunting a word that never comes	Every scanning loop checks for <code>'\0'</code> in its condition, not just for the separator
Pointers as a valid input — above all the empty list, whose head pointer is <code>NULL</code>	Dereferencing the head before checking there is one	An empty list is still a list: handle it wherever a list arrives. Otherwise let the subject decide: where it names a <code>NULL</code> case, return what it specifies; where it names none — a <code>libc</code> reimplementation, say — <code>NULL</code> is outside the contract, just as it is for the original

Category or situation	Classic bug	Defense
Numbers	<code>while (n > 0)</code> never runs, so 0 prints nothing	Make zero an explicit case whenever a loop peels digits
Numbers	2147483648 (<code>INT_MIN</code>)	Negating it overflows — <code>INT_MAX</code> is 2147483647
Numbers	Multi-sign string like <code>"-- +42"</code>	Stay in <code>int</code> : work on the negative side, whose range is the larger one, or deal with the one value that has no positive counterpart before any negation; test <code>INT_MIN</code> every time a number crosses your code
Memory	<code>malloc</code> returns <code>NULL</code>	Assuming <code>libc</code> rules when the subject defines its own, or vice versa
Memory	Array of strings ends	Using the result unchecked
<code>argv</code>	<code>./prog</code> alone (<code>argc == 1</code>)	Check every allocation; on failure, return what the subject specifies and, where the subject allows <code>free</code> , free anything already allocated
<code>argv</code>	<code>./prog ""</code>	Size for one extra slot and set the sentinel when the subject expects one
Output	Missing trailing newline	Check <code>argc</code> before touching <code>argv</code> ; take the no-argument behavior from the subject
Boundaries	Off-by-one at the terminator	Row one, applied to <code>argv[1]</code> ; test it explicitly
	Copy loop stops before writing <code>'\0'</code> , or writes it one byte past the buffer	Pipe every test through <code>cat -e</code> ; print the newline on every exit path where the subject shows one
		Allocate <code>length + 1</code> , write the terminator explicitly, and re-read every loop bound with the last index in mind

Appendix C — GDB and LLDB card

The command card from Debuggers: `gdb` and `lldb` as peers — one table, both debuggers, short forms in parentheses.

Task	<code>gdb</code>	<code>lldb</code>
Launch	<code>gdb ./prog</code>	<code>lldb ./prog</code>
Run with arguments	<code>run world o (r)</code>	<code>run world o (r)</code>
Break at function	<code>break strip_letter (b)</code>	<code>b strip_letter</code>
Break at file:line	<code>break strip_letter.c:19</code>	<code>b strip_letter.c:19</code>
Step over a line	<code>next (n)</code>	<code>next (n)</code>
Step into a call	<code>step (s)</code>	<code>step (s)</code>
Resume	<code>continue (c)</code>	<code>continue (c)</code>
Print a variable	<code>print j (p j)</code>	<code>p j</code>
Print in hex	<code>print/x len</code>	<code>p/x len</code>
Print a string	<code>print src</code>	<code>p src</code>

Task	<code>gdb</code>	<code>lldb</code>
Examine raw bytes	<code>x/16xb out</code>	<code>memory read --size 1 --format x --count 16 out</code>
All locals at once	<code>info locals</code>	<code>frame variable</code>
Call stack	<code>backtrace (bt)</code>	<code>bt</code>
Move up a frame	<code>up, frame 1</code>	<code>up, frame select 1</code>
Break on variable write	<code>watch j</code>	<code>watchpoint set variable j</code>
Show source here	<code>list</code>	<code>source list</code>
Quit	<code>quit (q)</code>	<code>quit (q)</code>

The five-step segfault triage, one line per step:

1. Reproduce the crash with the exact argument list that crashed.
2. Rebuild with `-g`, keeping `-Wall -Wextra -Werror` on.
3. Launch the debugger and `run` with those arguments.
4. `bt` — read the innermost frame in your file: the line, then the argument values (`0x0` there is the story half the time).
5. `print` the variables on that line; if the cause is still hidden, `break` one line earlier, rerun, and step with `next`.

A typical campus machine has `gdb` installed. 🏠 `lldb` is present too, on campus workstations and exam machines alike (checked at one campus). 🏠

Appendix D — Trace-reading card

The eight parts of a trace, in the order traces from two campuses show them; the exact format varies by campus and era. 🏠 The full walkthrough is Reading a trace, line by line.

Part	What it is	What you do with it
1	Host info: hostname, kernel, date, gcc and clang versions	Note which compiler versions judged you, and reproduce with the same one
2	<code>Collecting user files from Vogsphere plus your repo URL</code>	Confirms the grader saw your push — if this part looks wrong, the push never landed
3	The full git log of your exam repo	Check that your last commit is actually in it
4	<code>ls -lAR</code> of everything collected	Spot stray files — a leftover <code>a.out</code> or test <code>main.c</code> is visible right here
5	The verbatim compile line per exercise	Copy it into your terminal and run it on your file — reproduce the grader's compile gate exactly
6	Each test's exact invocation, with its <code>argv</code>	Re-run the failing case locally with the same arguments; the binary name is a random string, because the grader renames your program — nothing can depend on <code>argv[0]</code>
7	The diff verdict per test: <code>Diff OK :D</code> or the diff then <code>Diff KO :(</code>	Decode it: <code>-</code> lines are your output, <code>+</code> lines are the expected output, <code>\$</code> marks every line end. In a trace diff, a missing final newline shows as a <code>\ No newline at end of file</code> line; in your own <code>./prog cat -e</code> check, it shows as a missing <code>\$</code>

PartWhat it is

What you do with it

8

Grade: 0 or Grade: 1 per exercise, final grade at the end

Binary per exercise: the smallest mismatch costs the whole exercise

One caveat before you lean on this card: traces are the norm, not a promise — at least one campus account reports getting no clues at all about a failure, and a 2024 account says availability varies by exam. 🏠

Appendix E — Valgrind decoder

The tear-out card from the Valgrind section of Part II. The full invocation:

```
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./prog args
```

The four messages that matter:

Line in the report	What it means	First place to look
Invalid write of size 1	you wrote outside a block you own	allocation size math — the missing +1
Invalid read of size 1	you read outside a block you own	loop bounds; scanning past the '\0'
N bytes ... definitely lost	malloc'd, unreachable at exit, never freed	a free for every path out, including error paths — where the subject allows free
Conditional jump or move depends on uninitialised value(s)	you branched on a variable never assigned	the --track-origins=yes output names where it was born

And what clean looks like — the shape you want before every submit:

```
==29202== HEAP SUMMARY:
==29202==      in use at exit: 0 bytes in 0 blocks
==29202==    total heap usage: 1 allocs, 1 frees, 6 bytes allocated
==29202== All heap blocks were freed -- no leaks are possible
==29202== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

All heap blocks were freed and ERROR SUMMARY: 0 errors are the two lines that matter.

Appendix F — Master pool table

This is the guide's only pool tabulation — every exercise named anywhere in the chapters resolves to one row here. The classic pool is a 68-exercise enumeration from a 2016-era 42 Paris listing (source 7 in Appendix H, archived in 2025): 8 exercises at level 0, 14 at level 1, 15 at L2, 15 at L3, 9 at L4, and 7 at L5. That pool has been stable since about 2016, but pools evolve, campuses differ, and repositories disappear — expect new exercises on familiar archetypes, which is why the chapters teach archetypes, not answers. 🏠 The Level column is the classic baseline; where other sources place an exercise elsewhere, the disagreement lives in the Also-seen-at column. The Chapter column names the Part III chapter that teaches the archetype.

Concept keys: **A** raw write output · **B** string reimplementations · **C** argv char/word transforms · **D** argc/argv handling · **E** number-to-string and bases · **F** recursion and number theory · **G** malloc-returning arrays · **H** sorting and in-place arrays · **I** bit manipulation · **J** lists and function pointers · **K** hard parsers and algorithms.

Exercise	Level	Also seen at 🏠	Concept keys	Chapter
aff_a	L0	—	A	Level 0
aff_first_param	L0	—	A, D	Level 0
aff_last_param	L0	—	A, D	Level 0
aff_z	L0	—	A	Level 0
maff_alpha	L0	—	A	Level 0
maff_revalpha	L0	—	A	Level 0
only_z	L0	an <code>only_a</code> peer appears in other sources — see additions	A	Level 0
strlen_sh	L0	classic listing only; historical shell-era exercise, unlikely in modern pools 🏠	— (shell)	Level 0
first_word	L1	—	C	Level 1
ft_countdown	L1	L0 in most other sources 🏠	A, E	Level 0
ft_print_numbers	L1	L0 in most other sources 🏠	A, E	Level 0
ft_putstr	L1	—	A, B	Level 1
ft_strcpy	L1	—	B	Level 1
ft_strlen	L1	—	B	Level 1
ft_swap	L1	—	B (pointers)	Level 1
hello	L1	L0 in most other sources 🏠	A	Level 0
repeat_alpha	L1	—	C	Level 1
rev_print	L1	—	C	Level 1
rot_13	L1	—	C	Level 1
rotone	L1	—	C	Level 1
search_and_replace	L1	—	C	Level 1
ulstr	L1	—	C	Level 1
alpha_mirror	L2	—	C	Level 2
do_op	L2	—	D, E	Level 2

Exercise	Level	Also seen at 🏠	Concept keys	Chapter
ft_atoi	L2	—	E	Level 2
ft_strcmp	L2	—	B	Level 2
ft_strdup	L2	—	B, G	Level 2
ft_strrev	L2	—	B	Level 2
inter	L2	—	C	Level 2
is_power_of_2	L2	—	F, I	Level 2
last_word	L2	—	C	Level 2
max	L2	—	H	Level 2
print_bits	L2	—	I	Level 2
reverse_bits	L2	—	I	Level 2
swap_bits	L2	—	I	Level 2
union	L2	—	C	Level 2
wdmatch	L2	—	C	Level 2
add_prime_sum	L3	a real 2016 trace confirms L3	F	Level 3
epur_str	L3	—	C	Level 3
expand_str	L3	—	C	Level 3
ft_atoi_base	L3	—	E	Level 3
ft_list_size	L3	—	J	Level 3
ft_range	L3	—	G	Level 3
ft_rrange	L3	—	G	Level 3
hidenp	L3	—	C	Level 3
lcm	L3	—	F	Level 3
paramsum	L3	—	D, E	Level 3
pgcd	L3	—	F	Level 3
print_hex	L3	—	E	Level 3
rstr_capitalizer	L3	—	C	Level 3
str_capitalizer	L3	—	C	Level 3
tab_mult	L3	—	E	Level 3
fprime	L4	—	F	Level 4
ft_itoa	L4	—	E, G	Level 4
ft_list_foreach	L4	—	J	Level 4
ft_list_remove_if	L4	—	J	Level 4
ft_split	L4	—	G	Level 4

Exercise	Level	Also seen at 🏠	Concept keys	Chapter
rev_wstr	L4	—	C	Level 4
rostring	L4	—	C	Level 4
sort_int_tab	L4	—	H	Level 4
sort_list	L4	—	H, J	Level 4
brackets	L5	—	K	Level 5
brainfuck	L5	—	K	Level 5
check_mate	L5	L4 in several sources 🏠	K (2-D grid)	Level 5
ft_itoa_base	L5	—	E	Level 5
options	L5	—	I, K	Level 5
print_memory	L5	—	K (byte-level)	Level 5
rpn_calc	L5	—	K	Level 5

Attested additions. These are seen outside the classic enumeration — newer pools, other networks' campuses, or era-specific lists. They are never counted in the 68; treat each with its era in mind. 🏠

Exercise	Attested by and era	Level slot seen	Keys	Chapter
only_a	classic-era peer of <code>only_z</code> , attested in classic-era and later lists	L0	A	Level 0
fizzbuzz	newer — a 2024 campus list	L1-ish	A, E	Level 1
flood_fill	2018–2019 lists at L4, one 2019 list at L5; absent from the classic enumeration — level contested 🏠	L4/L5	K (2-D grid)	Level 4
biggest_pal	newer lists only (2019 and 2024)	L5	C, K	Level 5
cycle_detector	newer lists only (2019 and 2024)	L5	J	Level 5

Appendix G — Glossary

One line per term; anything asserted here is taught properly in the chapters.

- **additions** — exercises attested outside the classic pool, tabulated separately in Appendix F with their era; never counted in the 68. 🏠
- **archetype** — a family of exercises sharing one skill set; the guide teaches archetypes so you can solve pool exercises you have never seen.
- **cat -e** — prints output with a `$` at every line end, making missing newlines and trailing spaces visible.

- **cooldown** — a reported growing delay between `grademe` attempts. ⚠
- **diff (unified)** — the line-by-line comparison format the grader uses; `-` lines are yours, `+` lines are expected.
- **examshell** — the exam program you type `subject`, `grademe`, and `finish` into.
- **exercise** — one assignment; a function exercise is graded with the Moulinette's own `main`, a program exercise ships yours.
- **far side, the** — levels 6-10, known from a single 2017 account. ⚠
- **first-attempt ladder** — the final's per-level scoring: 9 points on the first try, 4 on the retry, 0 on the third, which only advances you.
- **gdb** — the GNU debugger; taught in Part II, tear-out card in Appendix C.
- **grademe** — the examshell command that triggers grading of what you pushed.
- **heap** — the memory pool `malloc` hands out, as opposed to your local variables; leaks live only here, but overflows can happen here or in a local array — and `valgrind` sees only the heap kind.
- **login** — the login-time command that authenticates you with your intra password.
- **leak** — heap memory you allocated and never freed; `valgrind` files it under "definitely lost", "indirectly lost", "possibly lost" or "still reachable", and only `All heap blocks were freed` means none of them.
- **libc** — the standard C library; the exam's early levels have you reimplement small slices of it.
- **lldb** — the LLVM debugger, taught as `gdb`'s peer; same card, Appendix C.
- **Moulinette** — 42's automated grader: it collects your push, compiles with warnings-as-errors, and diffs your output byte by byte.
- **NUL terminator** — the `'\0'` byte that ends every C string; everything string-shaped depends on it being there.
- **oracle** — a trusted existing program (`tr`, `rev`, `factor`) whose output you diff your own against; the method is in Testing yourself like the Moulinette does.
- **parser** — code that reads structured input, validates it, and turns it into data or actions; the heart of level 5.
- **pisciner** — a Piscine student; you.
- **pool** — the classic enumerated set of 68 exam exercises, tabulated once in Appendix F.
- **rendu/** — the git-tracked directory where your exam work must live.
- **sanitizer** — compiler-instrumented runtime checking (`-fsanitize=address,undefined`) that catches most out-of-bounds accesses and many kinds of undefined behavior, signed overflow among them — but only on the inputs you actually run, and not reads of uninitialised memory, which are `valgrind`'s to find.
- **segfault** — a crash from touching memory you don't own; a debugger turns it into a file and line number.
- **sentinel (NULL)** — the `NULL` pointer that ends a pointer array so a consumer knows where to stop.
- **subject** — the assignment text in `subject/`; the only authority on names, formats, and allowed functions.
- **trace** — the grader's report in `traces/`; anatomy in Appendix D.

- **undefined behavior** — an operation C gives no guaranteed result for (signed integer overflow, out-of-bounds access, use-after-free); it can look fine locally and fail on the grader's machine. Unsigned arithmetic wraps instead, and that is defined.
- **vacuous truth** — being correct by doing nothing; a loop over an empty string runs zero times, and that is often exactly right.
- **valgrind** — a memory-error and leak detector; decoder in Appendix E.
- **Vogsphere** — the internal git server the grader collects your files from.
- **-Werror** — the compiler flag that turns every warning into a compile failure; the grader uses it, so you compile with it.

Appendix H — Sources and trust methodology

There is no official documentation of this exam — its contents are under NDA — so every fact in this guide comes from community sources, and any of it can drift. The method used throughout: sourced beats unsourced; real grader artifacts outrank recollections; where accounts disagree, the guide shows the variance as variance (🏠) instead of picking a winner; and anything resting on a single account carries ⚠️. The list below is ranked by evidentiary weight.

1. **alibastida/exam_excercises** — the only real `.trace` files: verbatim compile lines, randomized binary names, `diff -U 3 ... | cat -e`, per-exercise Grade lines; mirrors the JCluzet subjects. (2016/2019 artifacts)
2. **JCluzet/42_EXAM**, the engine behind **grademe.fr** — the canonical practice emulator with the most current pool; its own Discussion #52 notes only the week-1/2 pools are exposed. (active 2026)
3. **Sbk3824/42-Piscine wiki**, "Final Exam tips" — the 9→4→0 ladder, "expect at least 11 questions", the `gcc -g -plus-debugger` workflow. (2018)
4. **Alex Kassil**, "Exam Final: Eight Hours of Fun" — the only first-hand walkthrough of levels 6–10. (2017)
5. **felixtanhm/42-piscine README** — the most recent logistics: registration, the 10-minute window, no norm, trace variability, vim over VS Code. (2024)
6. **codequoi**, "How to Prepare for the 42 Piscine" — reputable framing: duration, scoring. (2022)
7. **gcamerli/examshell** — the cleanest classic L0–5 enumeration; archived 2025. (2016-era content)
8. **ayoub0x1/C-Piscine-exam** — level-slot cross-check from the 1337 network. (2024)
9. **ilsyabri's blog** and **dev.to/lara_dev** — the `subject/`, `rendu/`, `traces/` layout and the unlock mechanics. (2024/2025)
10. **Oregonian in the Desert** (42 Abu Dhabi) — the cooldown ladder, the dissenting voice on traces, and the 8-exercises-at-1-point variant. (2021)
11. Weaker signals: **DeepWiki** (the 25/100 threshold) and **Victoria Nguyen** (the weekly ladder; the `-/+` trace convention). (2019 and later)

Also consulted: 42network.org, Manmeet2018/42-exam-miner, joaquim-oliveira-neto, VladlenaSkubi, emreakdik/42ExamPractice, and kristofk.com.